UCLA Samueli School of Engineering

Digital Tapeout: Physical Design

Text

My big ask: around 6:45, could someone go up a floor to the Faraday Room to ask them if they could leave the door open?

Logistics

Fall Quarter: Onboarding, Assignment & Capstone Completion

- Attend Lectures (thank you!)
- To join one of the subteams in the proceeding quarters, you must complete that team's subteam assignment. You can complete multiple assignments.
- Collaborate in a group of ~4 on the fall capstone, which will give you a holistic view of the process.

Winter, Spring Quarter: Design Cycle

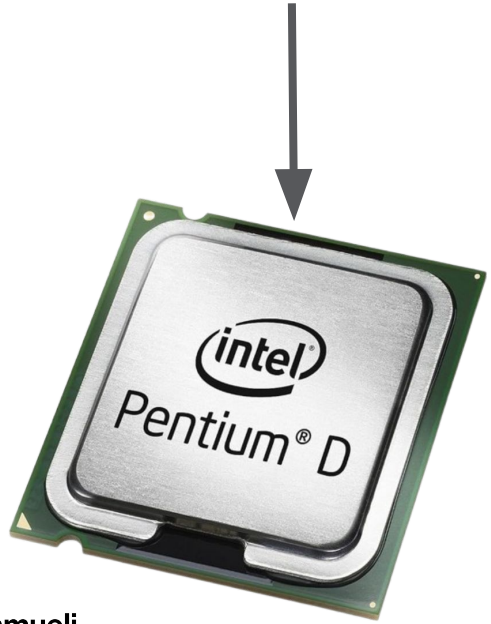
- Work with your assigned subteam to tapeout!
- Deviate from traditional IEEE project structure: **we expect you make this a big commitment of yours!**

Overview

1. Big Picture
2. Example Flow
3. Modes of Analysis
    a. Timing
    b. Physical
4. Case Studies
5. Server & NDA Info

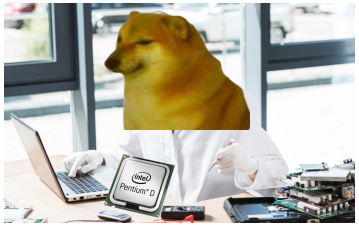
Big Picture

"I want a chip that can do A, B, and C"



Chip Designer

Chip Foundry



Bringup & Validation Engineer

Consumer

Big Picture

Chip Designer

Design Specification

Architectural Design

Digital Design

Functional Verification

RTL Description (HDL)

Frontend Design

Synthesis

Logic Equivalence Check

Gate Level Netlist

DFT

Floorplanning

Place and Route

Physical Verification & Signoff Checks

Tapeout

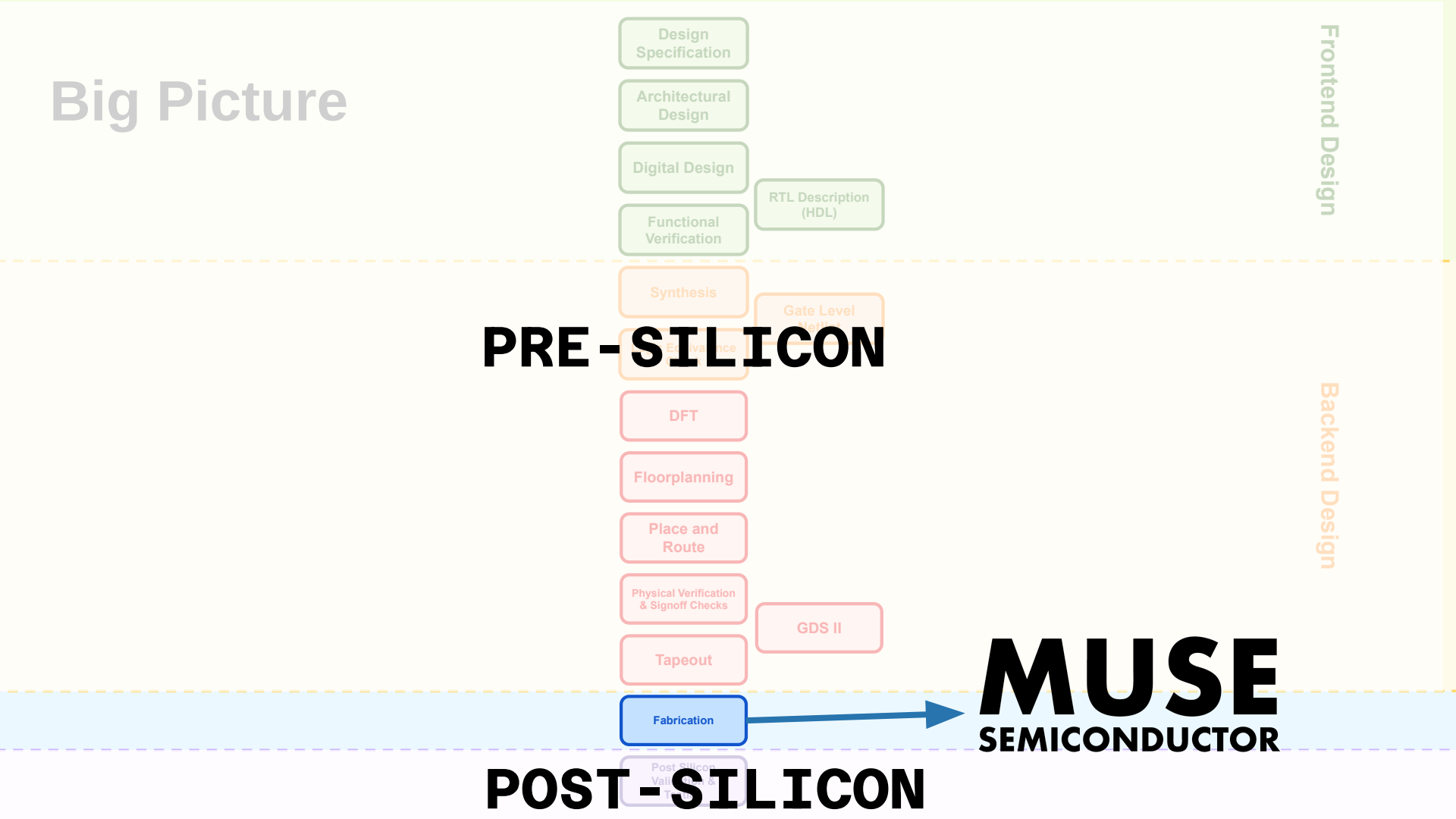
GDS II

Backend Design

Fabrication

Post Silicon Validation & Testing

Big Picture


Design Specification
Architectural Design
Digital Design
Functional Verification
RTL Description (HDL)
Frontend Design
Synthesis
Gate Level
DFT
Floorplanning
Place and Route
Physical Verification & Signoff Checks
GDS II
Tapeout
Backend Design
PRE-SILICON
Fabrication
MUSE
SEMICONDUCTOR
POST-SILICON

Design Specification

Architectural Design

Digital Design

RTL Description (HDL)

Functional Verification

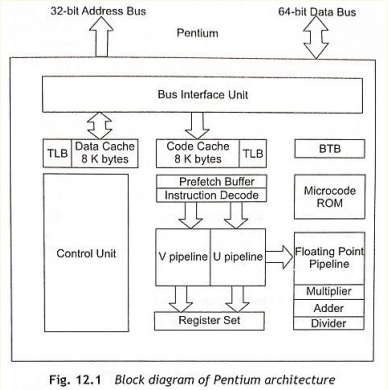


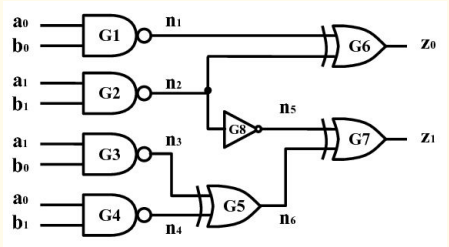
Fig. 12.1 *Block diagram of Pentium architecture*

```
module test (input clk, din, c1, c2, c3,
             output reg dout);

  always @(posedge clk) begin
    if (c1 & c2)
      if (~c2 & c3)
        dout <= din;
  end

endmodule
```

Synthesis

Logic Equivalence Check

Gate Level Netlist



DFT

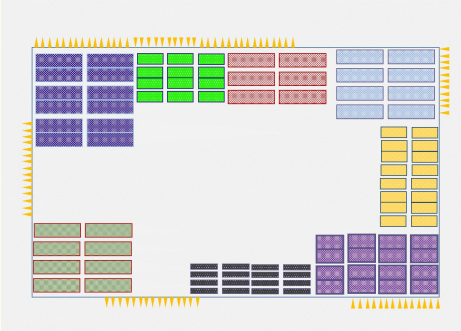
Floorplanning

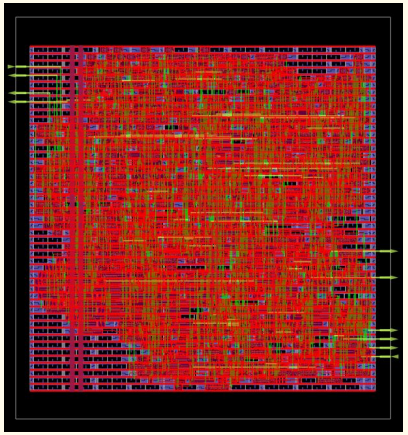
Place and Route

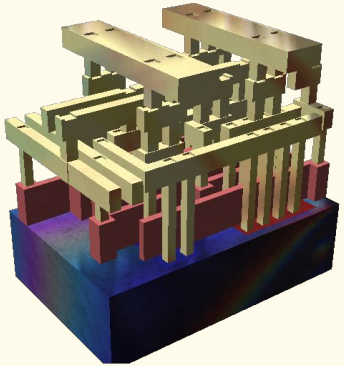
Physical Verification & Signoff Checks

Tapeout

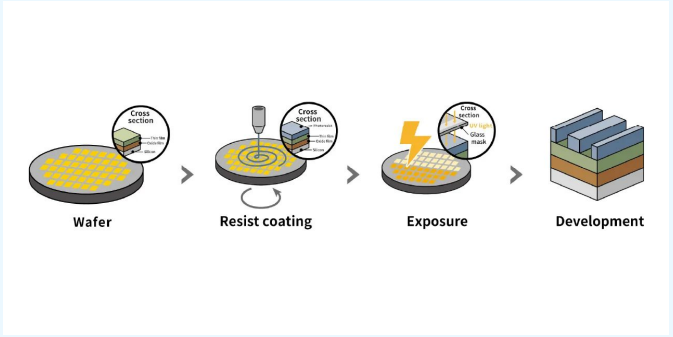
GDS II

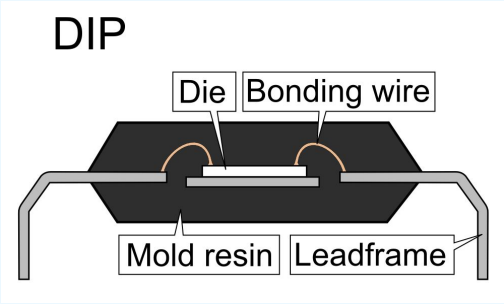




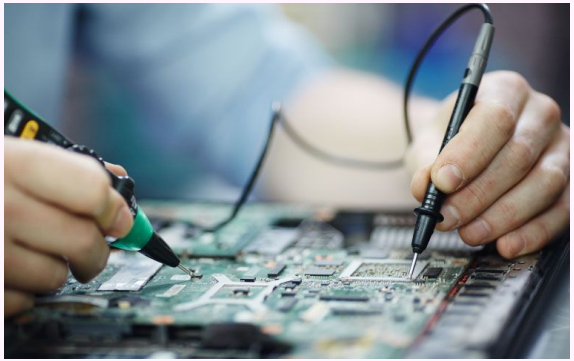


Fabrication





Post Silicon Validation & Testing



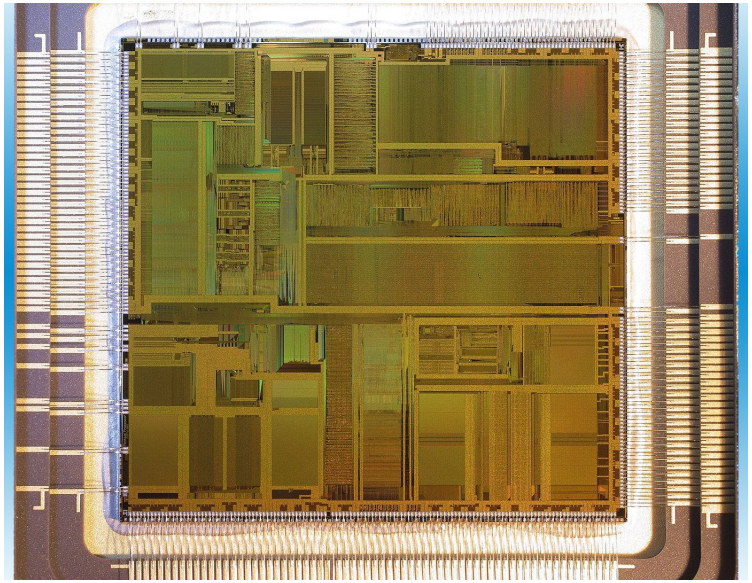
High Level to Low Level

Finished IC

High Level to Low Level

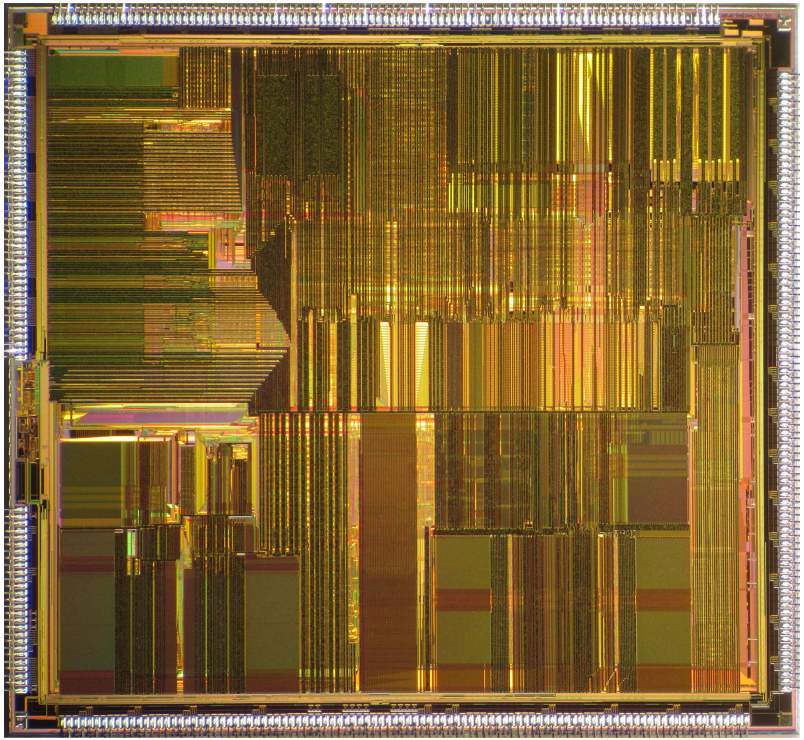
Inside the finished chip:
Registers, ICs
Packaging Styles: input

output pads w/ wires, take into programs
chip designers use



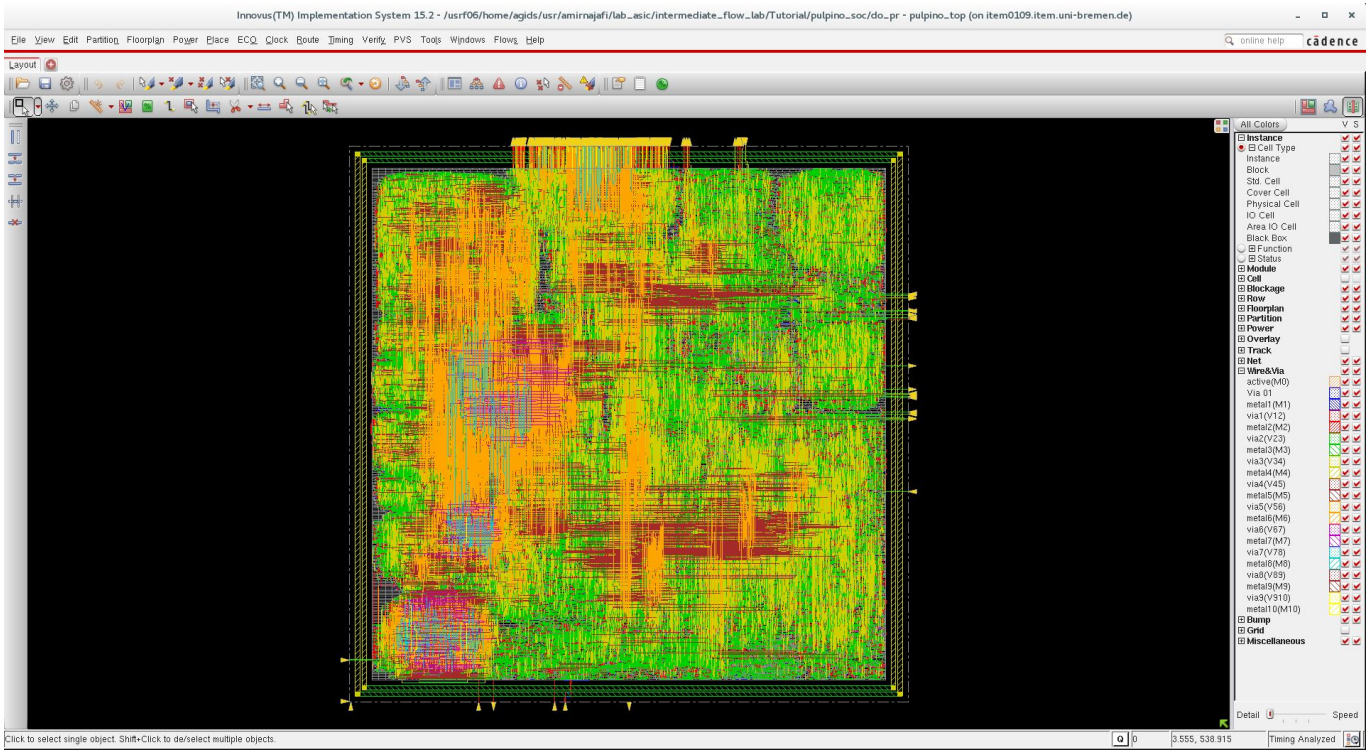
Packaging

High Level to Low Level



Die

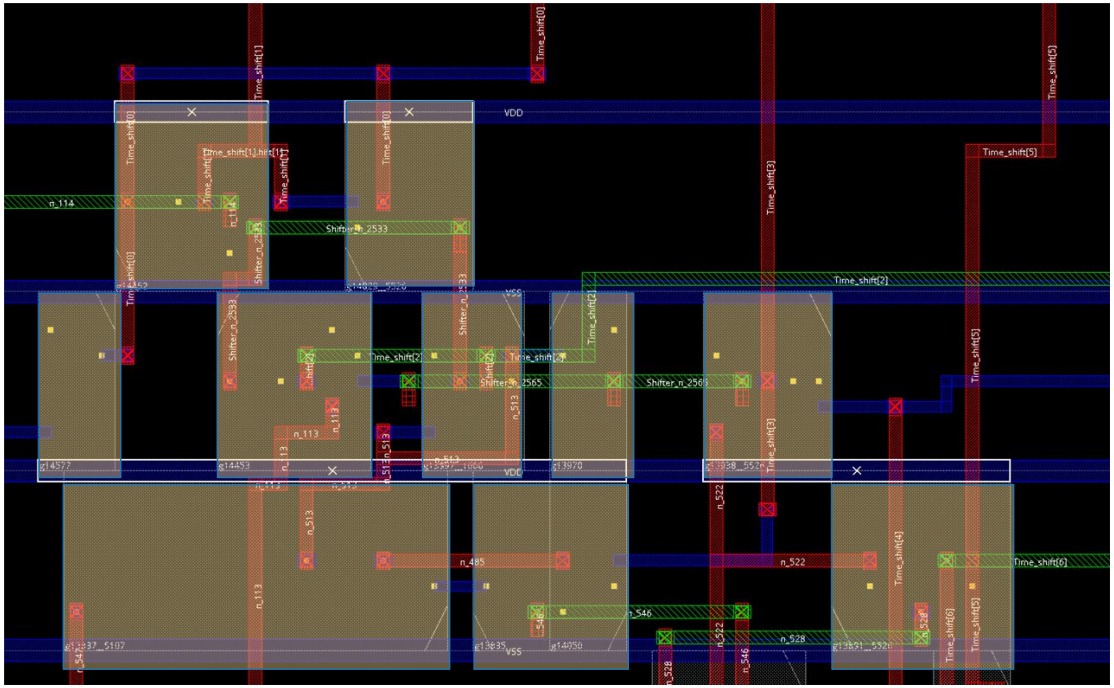
High Level to Low Level



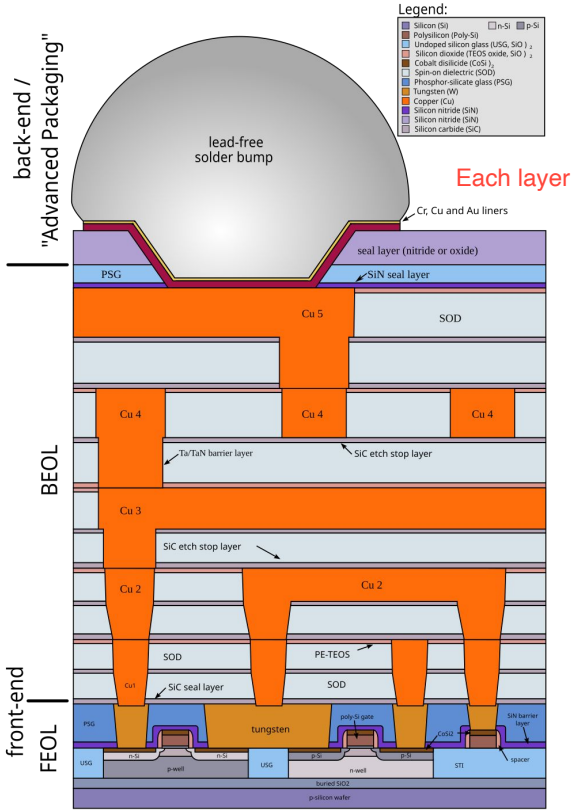
Die in Innovus

High Level to Low Level

wires labeled by metal layer by color, cells in yellow; zoom in next slides



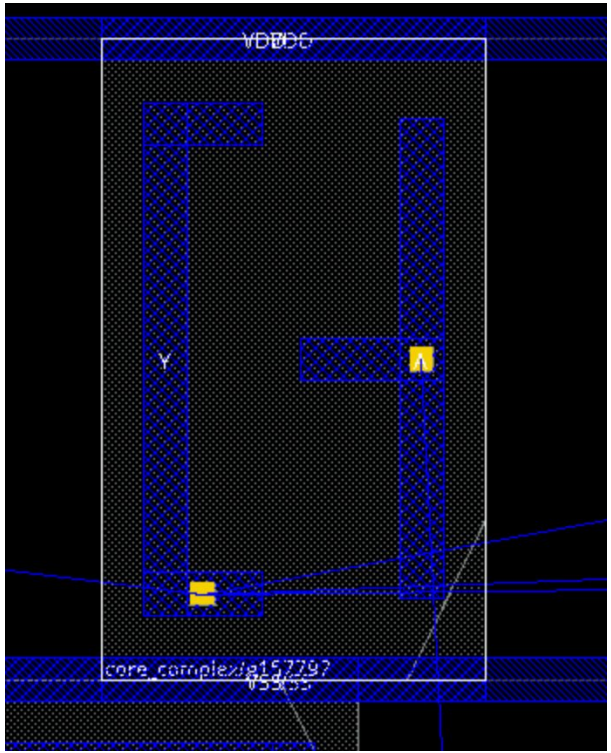
Cells & Wires



Metal Layers, Metal Stack

UCLA Samueli School of Engineering

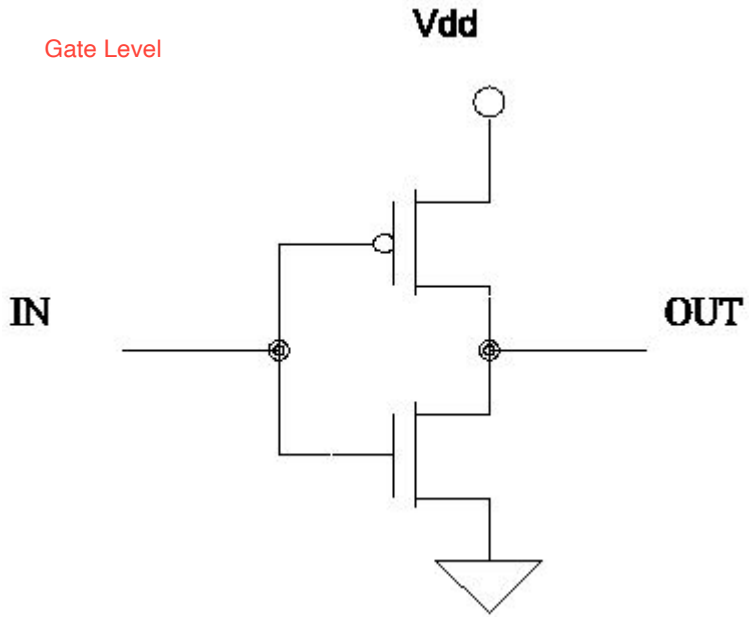
High Level to Low Level

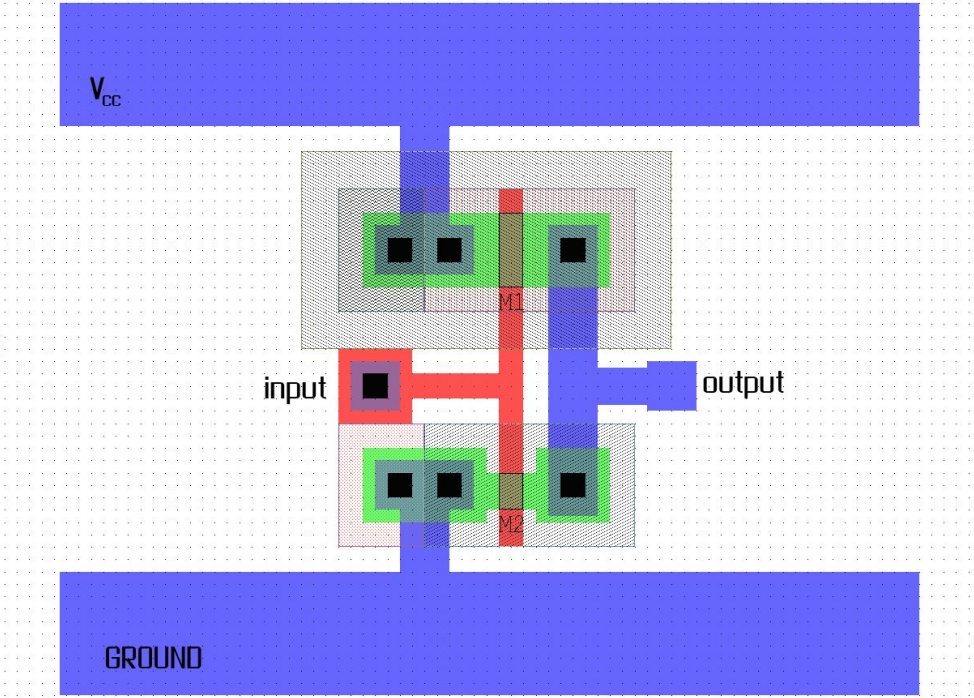


Cell View

High Level to Low Level

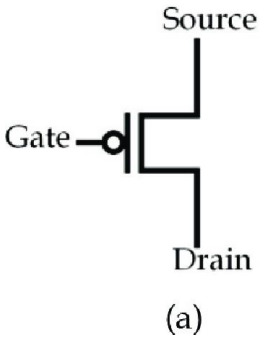
Gate Level



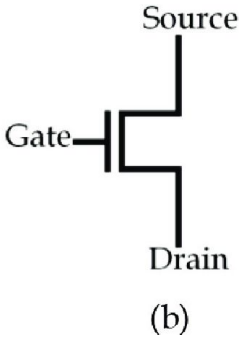


Cell View

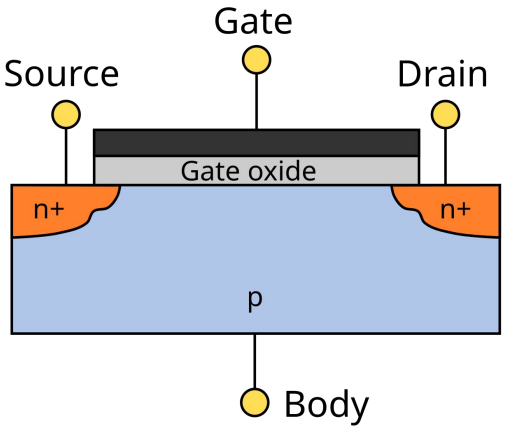
High Level to Low Level

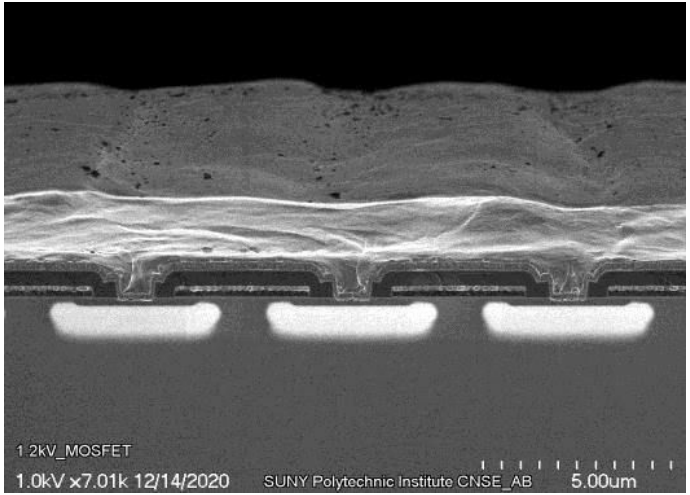


(a)



(b)





Transistor View

Transistors & Complementary MOSFETs

Transistor Level Behavior

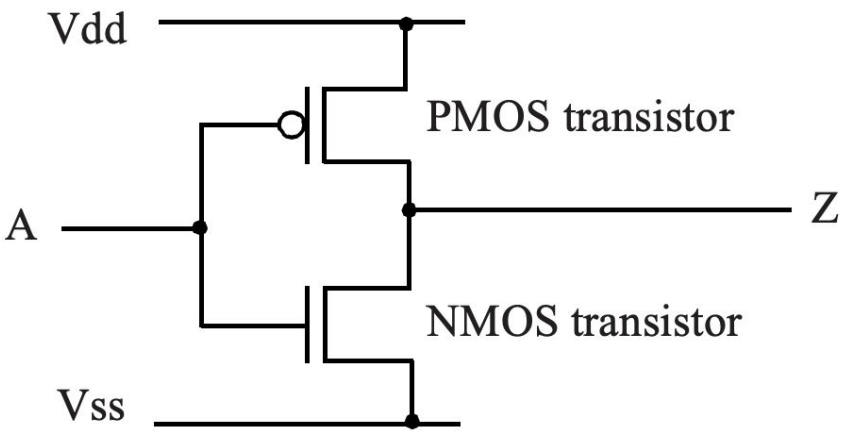
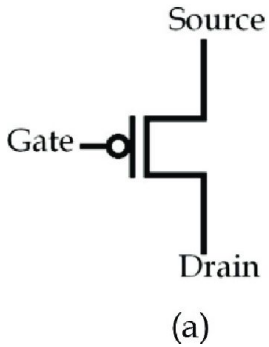
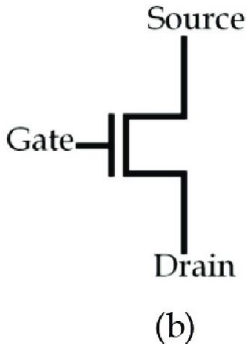


Figure 2-2 *A CMOS inverter.*

	Gate at 0	Gate at 1	Passes … well
NMOS	0	1	0
PMOS	1	0	1



(a)



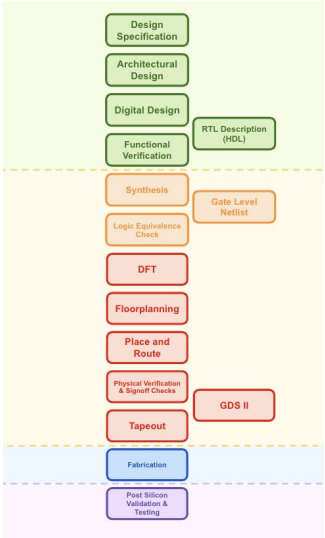
(b)

UCLA Samueli School of Engineering

For our purposes: MOSFETs are switches controlled by their gate voltage.

The Physical Design Roadmap

Flow Sequence



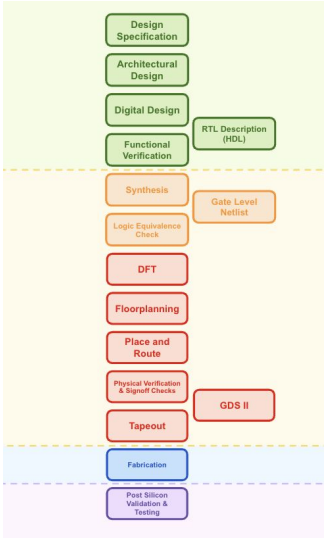
Setting Up the Flow: File Inputs & Outputs

Executing the Flow: Tool Knowledge

Interpreting the Results: Conceptual Understanding

The Physical Design Roadmap

Flow Sequence



.sv, .sdc, .lef, .lib, .techlef, .spef, .gds, .upf, .def, .mmmc

Innovus, Genus, PrimeTime, Calibre

Timing

Physical

Example Flow

Framing Device: Inputs to the Flow

Design Specific

netlist

Logical Description of Design

timing constraints

floorplan

power intent file

scan def

Technology Specific

standard cell library

physicality of each cell

timing of each cell

physical rules of the metal layers

Framing Device: Inputs to the Flow

Design Specific

netlist

timing constraints

floorplan

power intent file

scan def

Technology Specific

TECH LIB

standard cell library

physicality of each cell

timing of each cell

physical rules of the metal layers

An Aside: What's in a PDK?

Process Design Kit

- Open source vs. NDA PDKs

Libraries used for manufacture; we can use NDA PDKS w/ TSMC

- **.lib:** Liberty file - timing information. Internal pin capacitances, timing for each transition type.
- **.lef:** Library Exchange Format - physical blackbox of each standard cell layout to be used in place and route
- **.tlef, .techlef:** Technology Library Exchange Format - physical description of the metal layers, via cut layers,
- **.gds:** Graphic Design System files - binary file format to represent final physical layout of each standard cell
- **DRC Rule Deck**: a list of all manufacturing rules the final layout must abide by
- …among others!

Input File Examples

.lib

```
pin (Y) {
  direction : output;
  function : "(!A0 * !B0 * !C0) + (!A1 * !B0 * !C0)";
  min_capacitance : 0.001;
  power_down_function : "(!VDD) + (VSS)";
  related_ground_pin : VSS;
  related_power_pin : VDD;
  max_capacitance : 0.0138266;
  timing () {
    related_pin : "A0";
    timing_sense : negative_unate;
    timing_type : combinational;
    cell_rise (delay_template) {
      index_1 ("0.006, 0.0129266, 0.0278495, 0.06, 0.129266, 0.278495, 0.6");
      index_2 ("0.001, 0.00145532, 0.00211797, 0.00308232, 0.00448578, 0.00652826, 0.00950072, 0.0138266");
      values ( \
        "0.193016, 0.203514, 0.218715, 0.240738, 0.272674, 0.318939, 0.386085, 0.48353", \
        "0.195517, 0.206039, 0.221274, 0.243346, 0.275319, 0.321642, 0.388817, 0.486288", \
        "0.201855, 0.212435, 0.227748, 0.249899, 0.281959, 0.328377, 0.395615, 0.493243", \
        "0.215, 0.225636, 0.241039, 0.263312, 0.295507, 0.342089, 0.409518, 0.507258", \
        "0.239522, 0.250221, 0.265637, 0.287914, 0.320193, 0.36682, 0.434409, 0.532324", \
        "0.282562, 0.293725, 0.309909, 0.332677, 0.365118, 0.411777, 0.479289, 0.577442", \
        "0.337799, 0.351239, 0.370721, 0.397989, 0.435417, 0.488845, 0.562666, 0.664109" \
      );
    }
    rise_transition (delay_template) {
      index_1 ("0.006, 0.0129266, 0.0278495, 0.06, 0.129266, 0.278495, 0.6");
      index_2 ("0.001, 0.00145532, 0.00211797, 0.00308232, 0.00448578, 0.00652826, 0.00950072, 0.0138266");
      values ( \
        "0.155336, 0.169732, 0.190614, 0.220877, 0.264951, 0.328913, 0.421867, 0.556864", \
        "0.155373, 0.169736, 0.190621, 0.220921, 0.264941, 0.328912, 0.421746, 0.556864", \
        "0.155384, 0.169755, 0.190567, 0.220917, 0.264937, 0.328864, 0.42182, 0.556801", \
        "0.155414, 0.169699, 0.190613, 0.220908, 0.264928, 0.328861, 0.421756, 0.556804", \
        "0.155782, 0.170039, 0.19081, 0.221003, 0.26501, 0.328823, 0.421782, 0.556859", \
        "0.167967, 0.181332, 0.200692, 0.229262, 0.271115, 0.332759, 0.423439, 0.557152", \
        "0.208577, 0.221914, 0.242701, 0.271331, 0.312319, 0.37116, 0.45481, 0.579885" \
      );
```

.lef

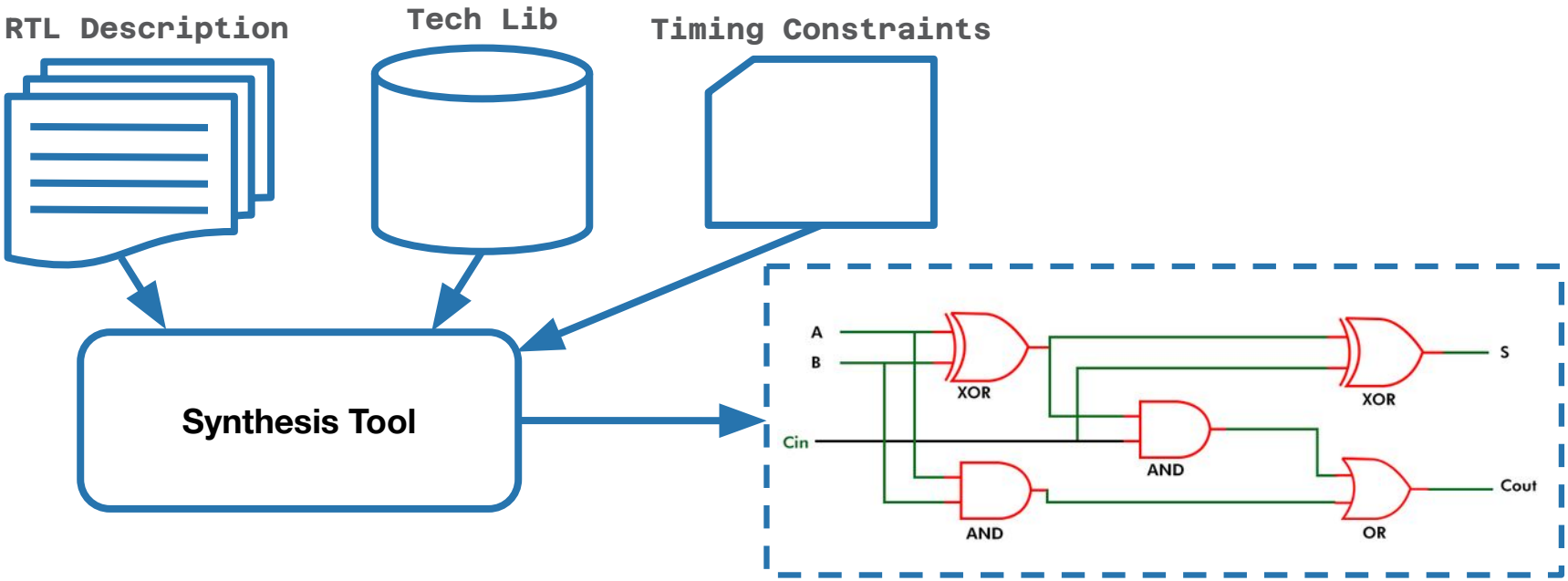
```
MACRO AOI211X1
  CLASS CORE ;
  ORIGIN 0 0 ;
  FOREIGN AOI211X1 0 0 ;
  SIZE 2.76 BY 4.14 ;
  SYMMETRY X Y ;
  SITE CoreSite ;
  PIN A0
    DIRECTION INPUT ;
    USE SIGNAL ;
    ANTENNAMODEL OXIDE1 ;
      ANTENNAGATEAREA 0.2835 LAYER met1 ;
      ANTENNAMAXAREACAR 0.308818 LAYER met1 ;
      ANTENNAMAXSIDEAREACAR 0.363316 LAYER met1 ;
    PORT
      LAYER met1 ;
        RECT 0.79 1.05 1.05 2.29 ;
    END
  END A0
  PIN A1
    DIRECTION INPUT ;
    USE SIGNAL ;
    ANTENNAMODEL OXIDE1 ;
      ANTENNAGATEAREA 0.2835 LAYER met1 ;
      ANTENNAMAXAREACAR 0.308818 LAYER met1 ;
      ANTENNAMAXSIDEAREACAR 0.363316 LAYER met1 ;
    PORT
      LAYER met1 ;
        RECT 0.33 1.57 0.59 2.81 ;
    END
  END A1
  PIN B0
    DIRECTION INPUT ;
    USE SIGNAL ;
    ANTENNAMODEL OXIDE1 ;
      ANTENNAGATEAREA 0.2835 LAYER met1 ;
      ANTENNAMAXAREACAR 0.326808 LAYER met1 ;
      ANTENNAMAXSIDEAREACAR 0.38448 LAYER met1 ;
    PORT
      LAYER met1 ;
        RECT 1.25 1.57 1.51 2.81 ;
    END
```

.tlef

```
MACRO AND2X2
  CLASS CORE ;
  ORIGIN 0 0 ;
  FOREIGN AND2X2 0 0 ;
  SIZE 0.95 BY 2.47 ;
  SYMMETRY X Y ;
  SITE CoreSite ;
  PIN A
    DIRECTION INPUT ;
    USE SIGNAL ;
    PORT
      LAYER metal1 ;
        RECT 0.1475 0.8475 0.2225 0.9825 ;
    END
  END A
```

UCLA Samueli School of Engineering

Synthesis



Begin Actual Flow, Need to convert RTL to gate-level netlist
Use Tech Lib for standard cells

Gate-Level Netlist

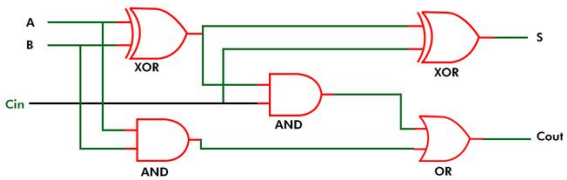
Synthesis

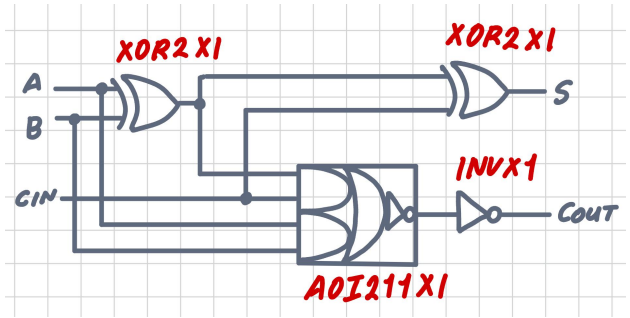
Set library, RTL paths

Elaborate → **Synthesis (Generic)** → **Synthesis (Map)**

```
`include "constants.vams"
`include "disciplines.vams"
`timescale 1ns/1ps

module verAMS_array_delay_v2 (IN, OUT);
            input IN;
            //electrical IN;
            wire IN; //Analog port connections in array

            output [0:1] OUT;
            logic [0:1] OUT;
            wire [0:1] OUT;

            genvar i;
            real tdelay_ns;

for (i=0; i<2; i=i+1)
begin
//tdelay_ns = (10*i) + 10;
//verAMS_delay #(.tdelay(tdelay_ns)) (IN, OUT[i]);
verAMS_delay #(.tdelay(30)) (IN, OUT[i]);
end

/* This works.
verAMS_delay #(.tdelay(30)) DUT0 (IN, OUT[0]);
verAMS_delay #(.tdelay(20)) DUT1 (IN, OUT[1]);
*/
endmodule
```

```
`include "constants.vams"
`include "disciplines.vams"
`timescale 1ns/1ps

module verAMS_delay (IN, OUT );
    input IN;
    electrical IN;

    output OUT;
    logic OUT;
    reg OUT;

    parameter real tdelay  = 20;

initial OUT = 0;

always @(cross(V(IN)-0.6,+1))
    begin
    OUT <= #tdelay 1;
    end
always @(cross(V(IN)-0.6,-1))
    begin
    OUT <= #tdelay 0;
    end
endmodule
```





Logic Equivalence Check

UCLA Samueli School of Engineering

Synthesis

Set library, RTL paths

```
set_db init_lib_search_path <lib_search_path>
set_db init_hdl_search_path <hdl_search_path>
set_db library [glob -directory [get_db init_lib_search_path] *ss*.lib]
read_hdl -sv [glob -directory [get_db init_hdl_search_path] *.sv]
```

Elaborate

```
elaborate <top_module>
```

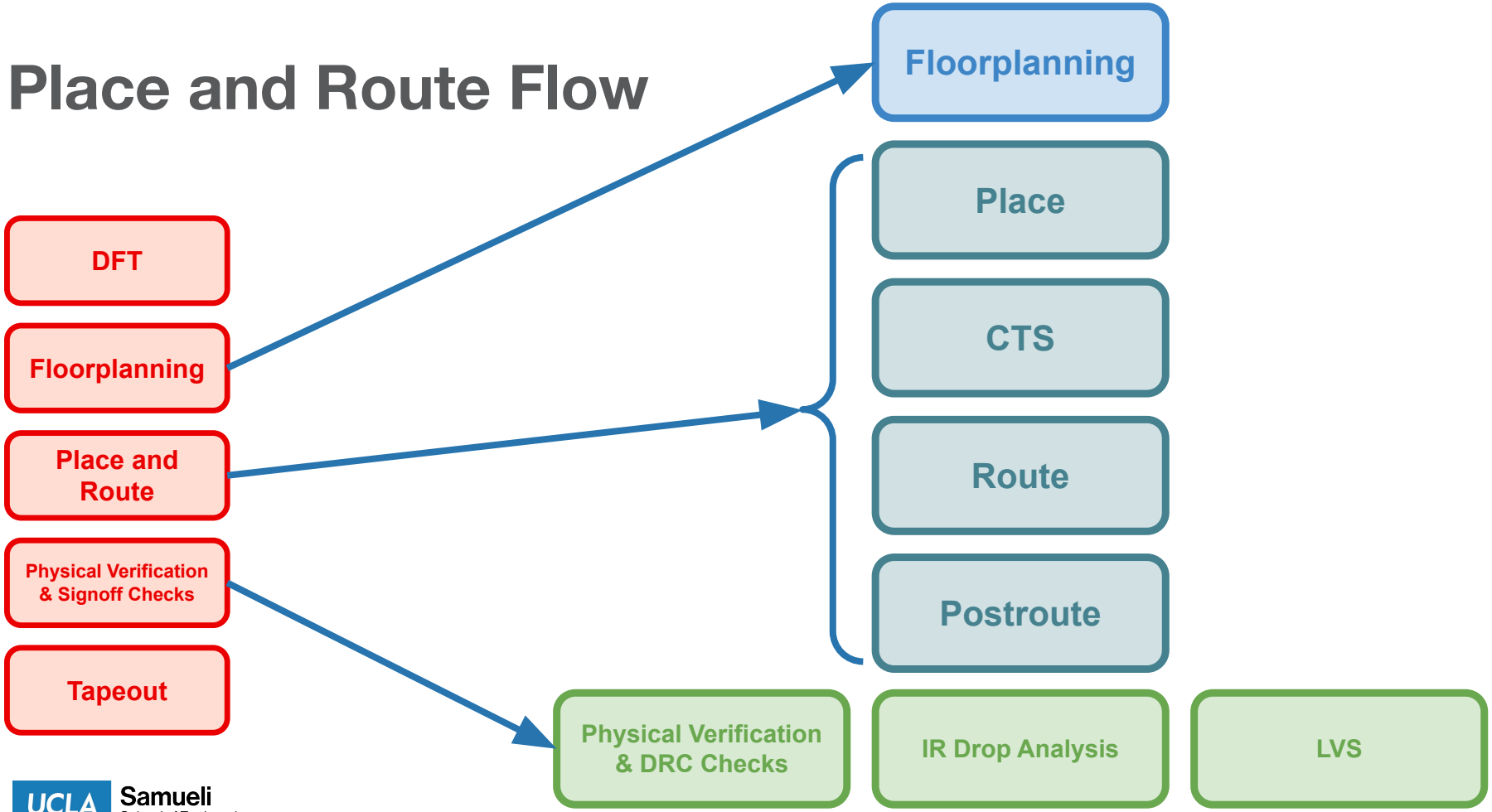
Synthesis (Generic)

```
syn_generic
```

Synthesis (Map)

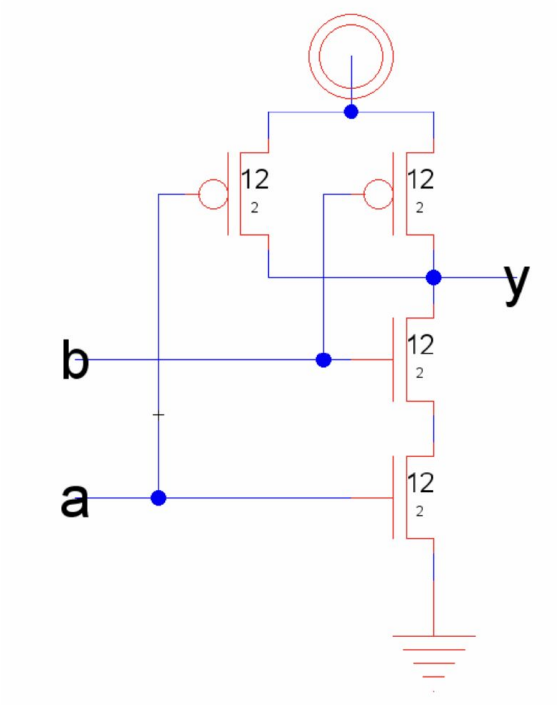
```
syn_map
```

Place and Route Flow



Building Blocks: Standard Cells

schematic



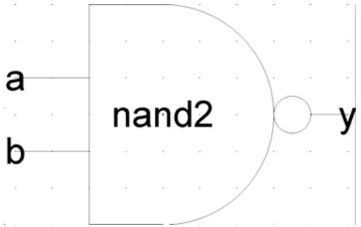


Figure 3: nand2{ic} final version

logic symbol

layout of standard cell

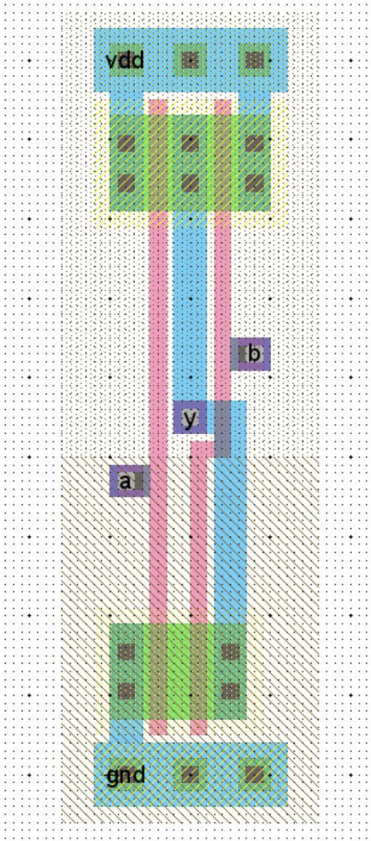
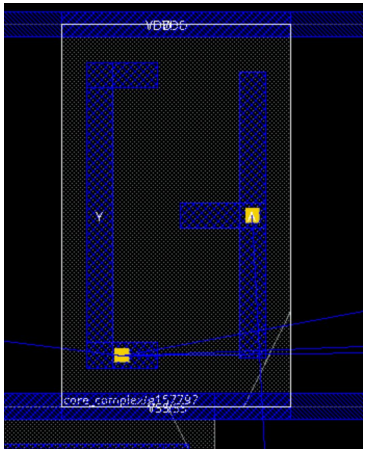
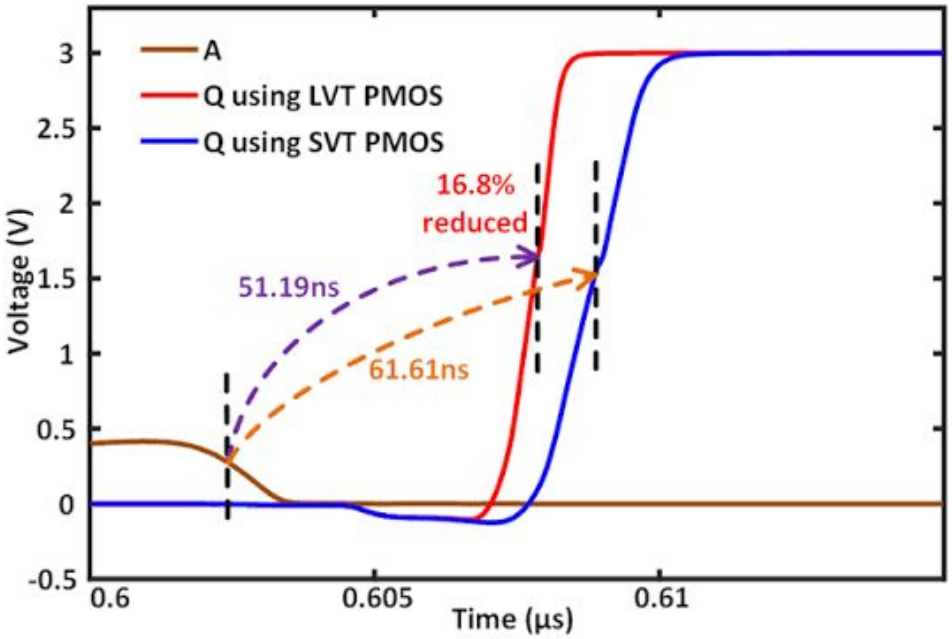


Figure 7: nand2{lay}

blackboxed cell



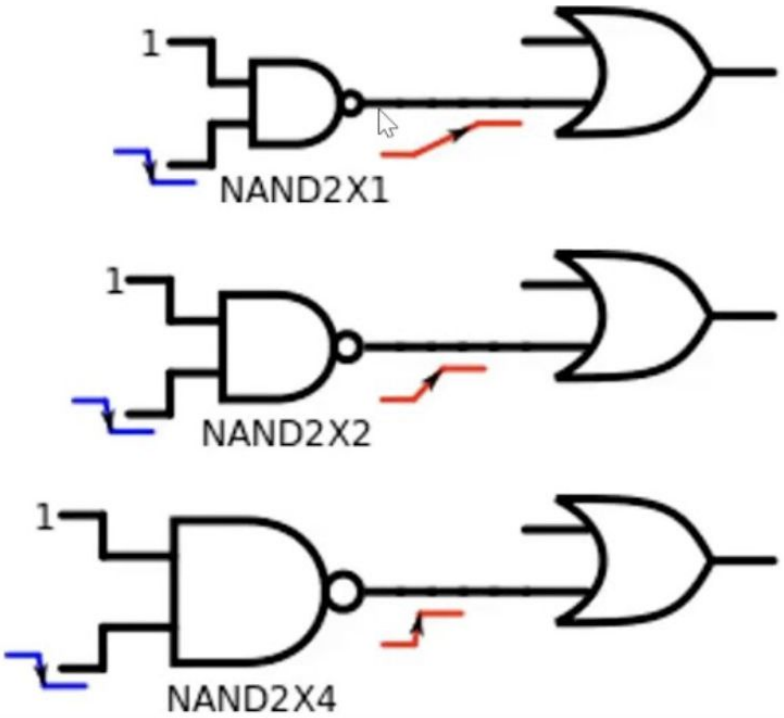
Cell Flavors: Voltage Threshold, VT Cells



+ Lower voltage threshold means cell will switch faster for the same input signal.
+ Faster propagation delay.
+ Same cell footprint as other VT cells.

- Lower voltage threshold means leakage power will be greater.

Cell Flavors: Drive Strength, X# Cells



+ Wider channel means max current through transistor can be greater, higher drive strength.
+ Can drive large fanout (capacitive load) to prevent transition violations and ensure timing is met.

- Larger cell means higher cell area.
- Higher switching current means greater dynamic power consumption.

Building Blocks: Timing Constraints

Define the timing characteristics of your design.

Netlists can only convey so much timing information. **A designer chooses these values, and tweaks them based on feedback.** Questions to ask:

1. What is the desired clock period of the design? `create_clock`
2. Are there any delays from external sources that need to be accounted for? `set_input_delay / set_output_delay`
3. What is my allowable clock uncertainty (accounts for skew, jitter, etc)? `set_clock_uncertainty`
4. Are there any false timing paths? `set_false_path`
5. Are there any multicycle timing paths? `set_multicycle_path`
6. What is my max allowable fanout? `set_max_fanout`

Need to specify/constraints in Synthesis tool for what the design will actually do

Let's Review: Main Inputs Into PnR

Gate Level Netlist

Post-Synthesis, Logic Equivalence Check Passed

Timing Constraints (.sdc)

Clock period, desired skew, fanout, IO delays specified

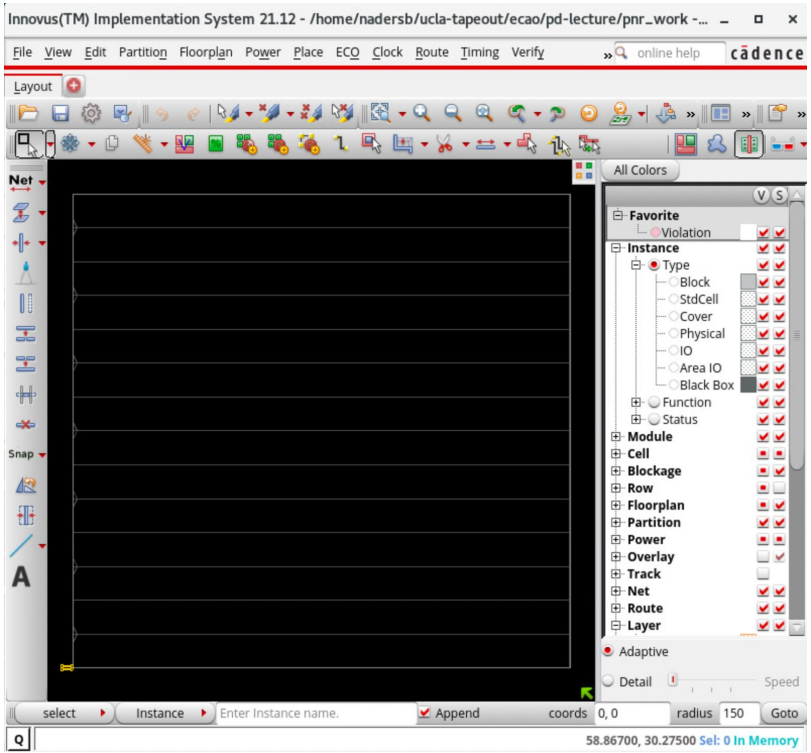
Timing Library (.lef, .lib, .techlef, etc)

Design rules and standard cells from foundry

Setup

Tool used is Innovus

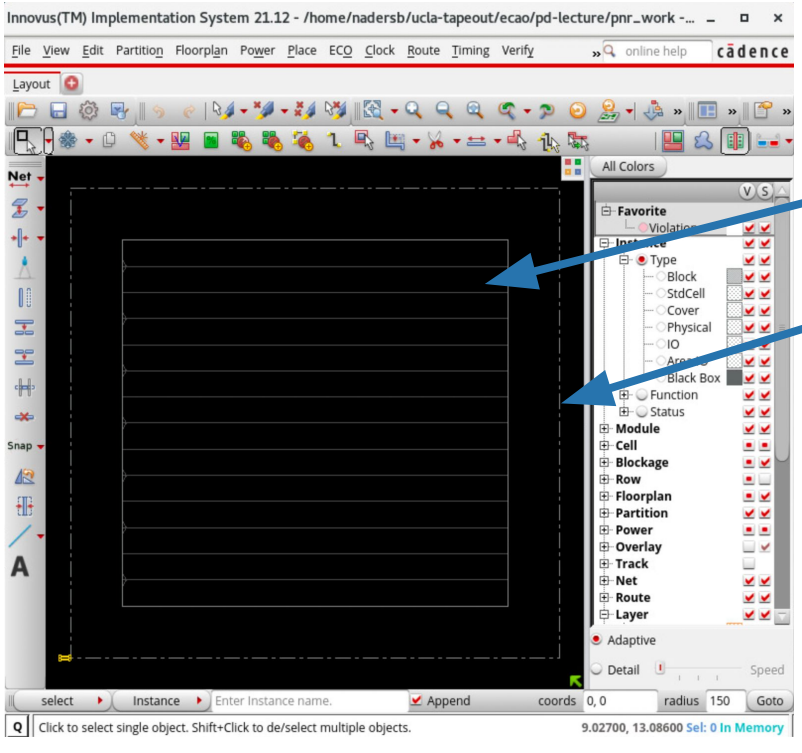
Load in tech library, netlist, timing constraints. Initialize the design with slow and fast timing corners.




```
# set initial variables
set init_verilog  <path>
set init_top_cell  "RCA_16_bit"

# lef files - always put the tech lef first!
set init_lef_file <pointers to tech lef and lef files>
set init_gnd_net   "VSS"
set init_pwr_net   "VDD"

# MCMM setup
# constraint mode (sdc)
create_constraint_mode -name CON -sdc_file <path to sdc>

# create slow case delay corner
create_library_set -name WC_LIB_SLOW -timing <ss corner lib path>
create_rc_corner -name _slow_rc_corner_ -T 125
create_delay_corner -name WC_SLOW -library_set WC_LIB_SLOW -rc_corner
_slow_rc_corner_

# create slow case analysis view
create_analysis_view -name WC_SLOW_VIEW -delay_corner WC_SLOW -constraint_mode
CON

# create fast case delay corner
create_library_set -name WC_LIB_FAST -timing <ff corner lib path>
create_rc_corner -name _fast_rc_corner_ -T -0
create_delay_corner -name WC_FAST -library_set WC_LIB_FAST -rc_corner
_fast_rc_corner_

# create fast case analysis view
create_analysis_view -name WC_FAST_VIEW -delay_corner WC_FAST -constraint_mode
CON

# analyze timing using the analysis views we made
init_design -setup {WC_SLOW_VIEW}  -hold {WC_FAST_VIEW}
```

Floorplanning, IOs

Size the floorplan, define core and die area dimensions, site row creation, macro placement (if applicable)

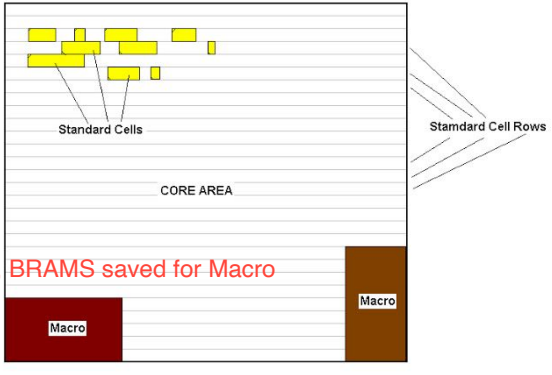


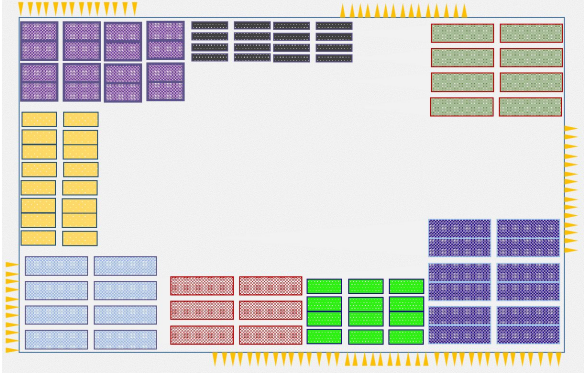

```
floorPlan -r 1.0 0.70 8.0 8.0 8.0 8.0
```

Core area

Die area

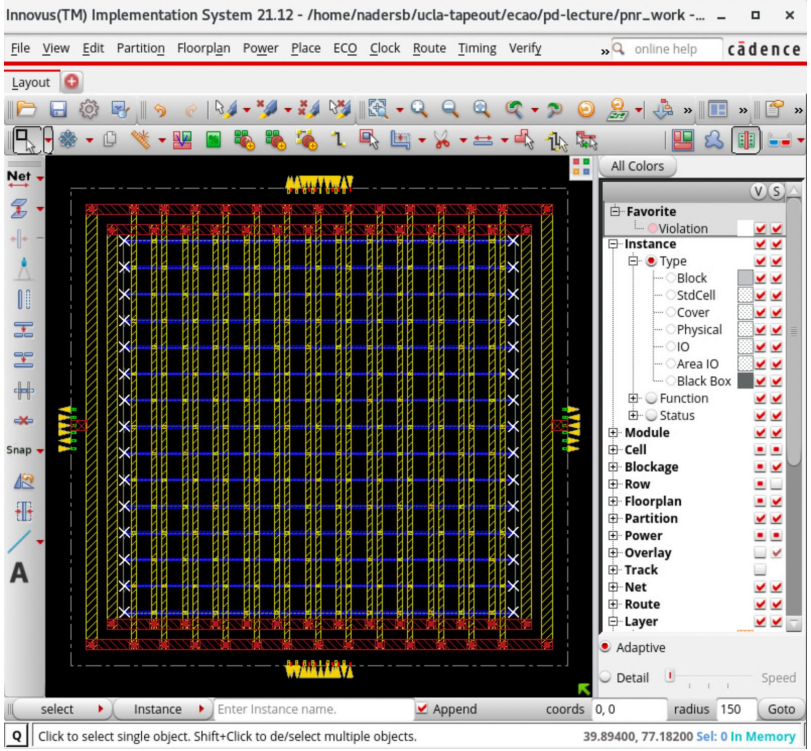
DRAMS, SRAMS, BRAMS saved for Macro



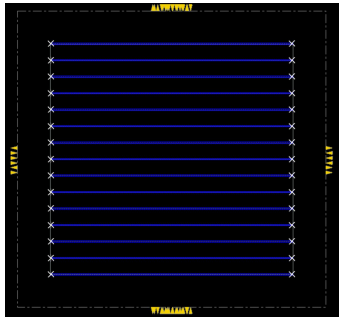


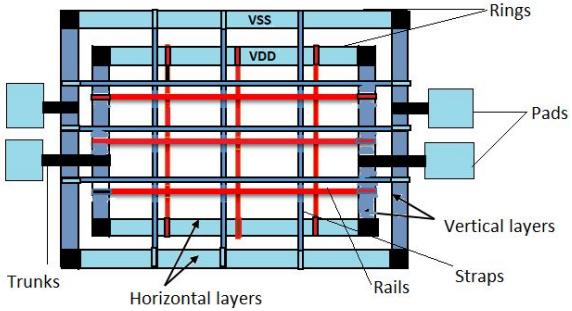
Power Distribution Network

Create power rings and stripes for the power distribution network.




```
sroute -nets {VDD VSS} -connect {corePin}
addRing \
  -nets {VDD VSS} -width 1.6 -spacing 1.6 \
  -layer [list top 5 bottom 5 left 4 right 4]
addStripe \
  -nets {VSS VDD} -layer 5 -direction horizontal \
  -width 0.8 -spacing 4.8 \
  -set_to_set_distance 11.2 -start_offset 2.4
addStripe \
  -nets {VSS VDD} -layer 4 -direction vertical \
  -width 0.8 -spacing 2.4  \
  -set_to_set_distance 4.8 -start_offset 1.2
```



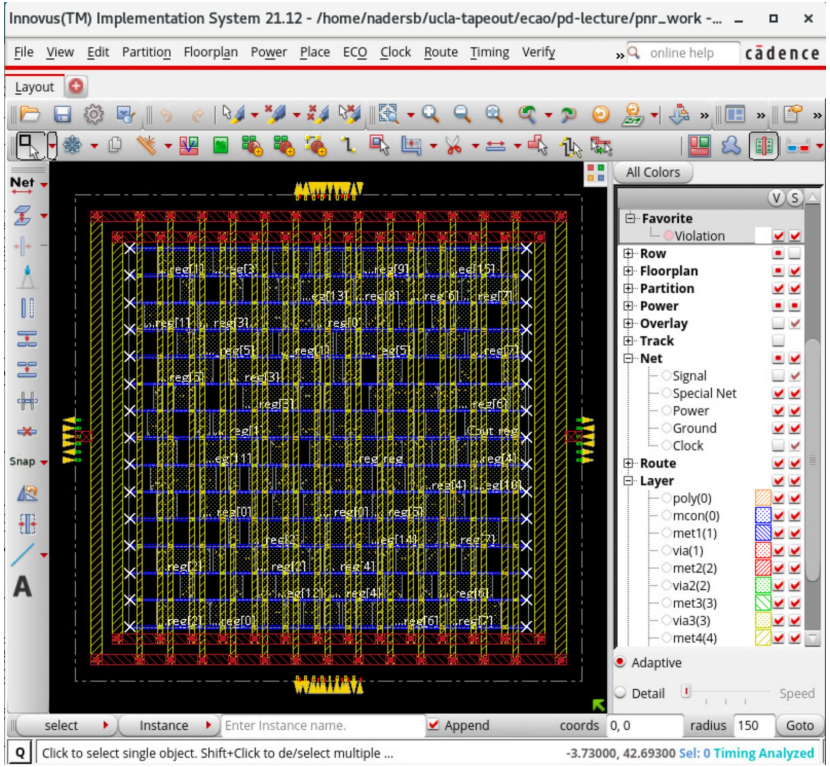


UCLA Samueli School of Engineering

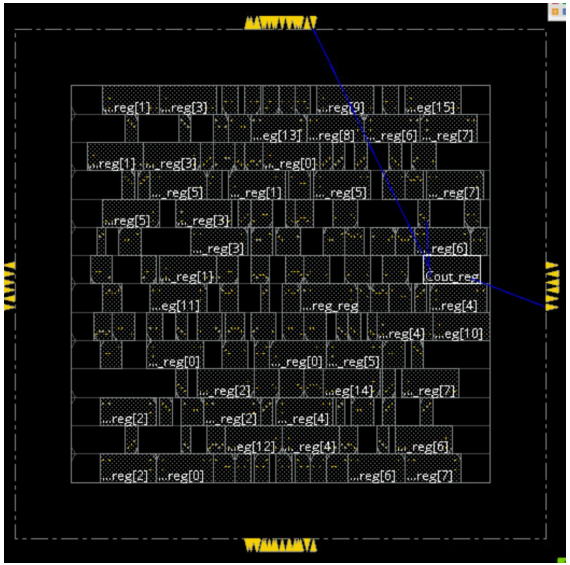
The lower resistance, the better the voltage is the same around these areas, higher layer so less resistance (thicker models)

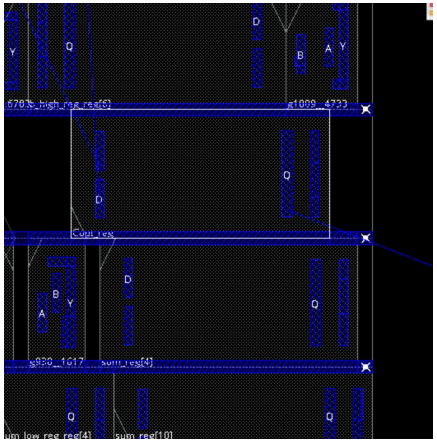
Placement of Standard Cells

Standard cells are placed in site rows and legalized. Power rails are hooked up.




```
place_opt_design
```





utilization

Physical Only Cells

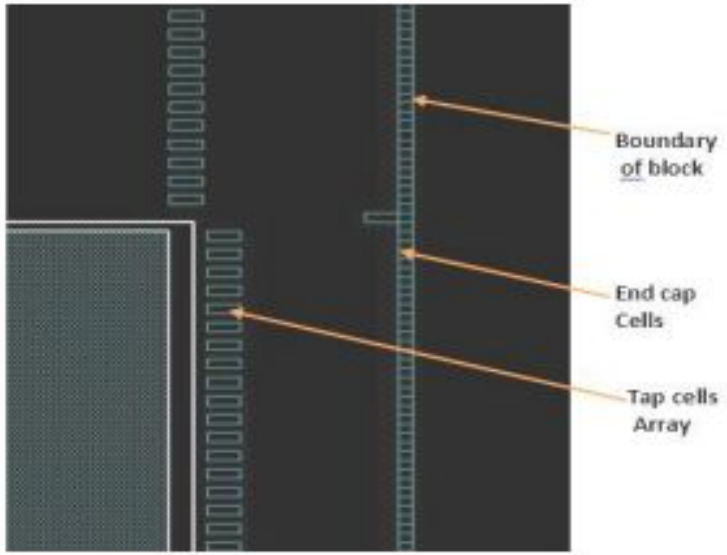
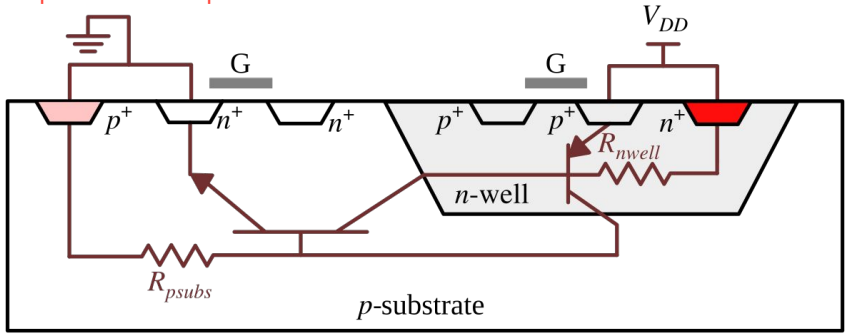


fig: TAP Cells and END CAP Cells

```
addWellTap -cell TAPCELL_ASAP7_75t_L -cellInterval 25.920
-inRowOffset 1.296
```

Latchup: some sort of parasitic circuit can form and cause the entire structure to fail



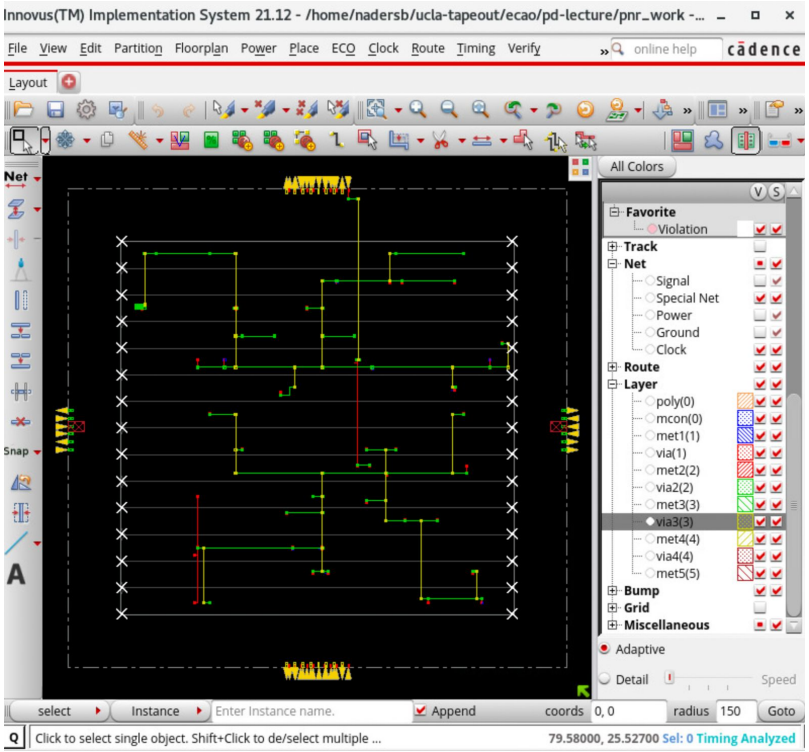
Tap Cells are placed at even intervals

Tap cells, Edge cells, Decaps, Fillers, Spare Cells

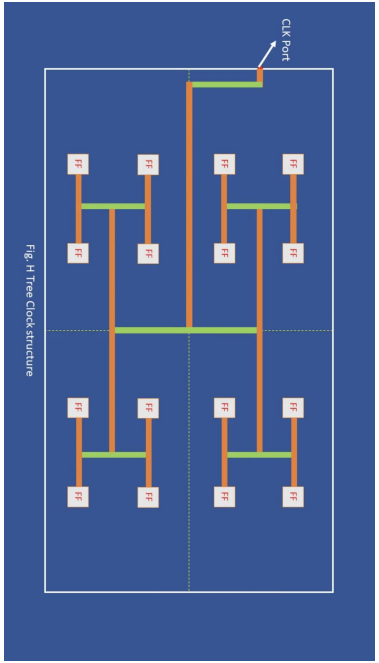
Physical-Only Cells which have no logical function and are used to satisfy physical or manufacturing requirements.

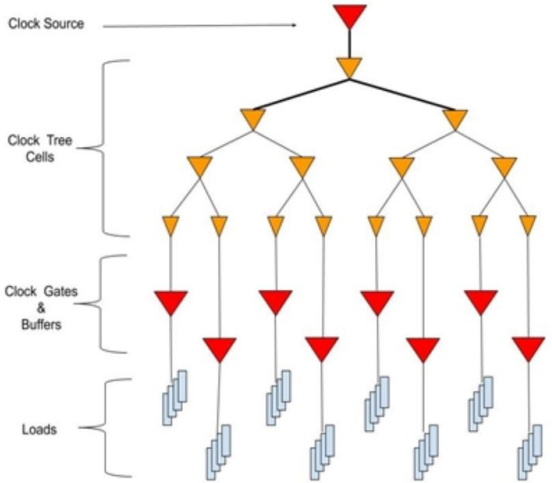
Clock Tree Synthesis

Route clock nets, inserting buffers to break up high fanout clock nets, to deliver clock signal to all sinks (sequential cells)



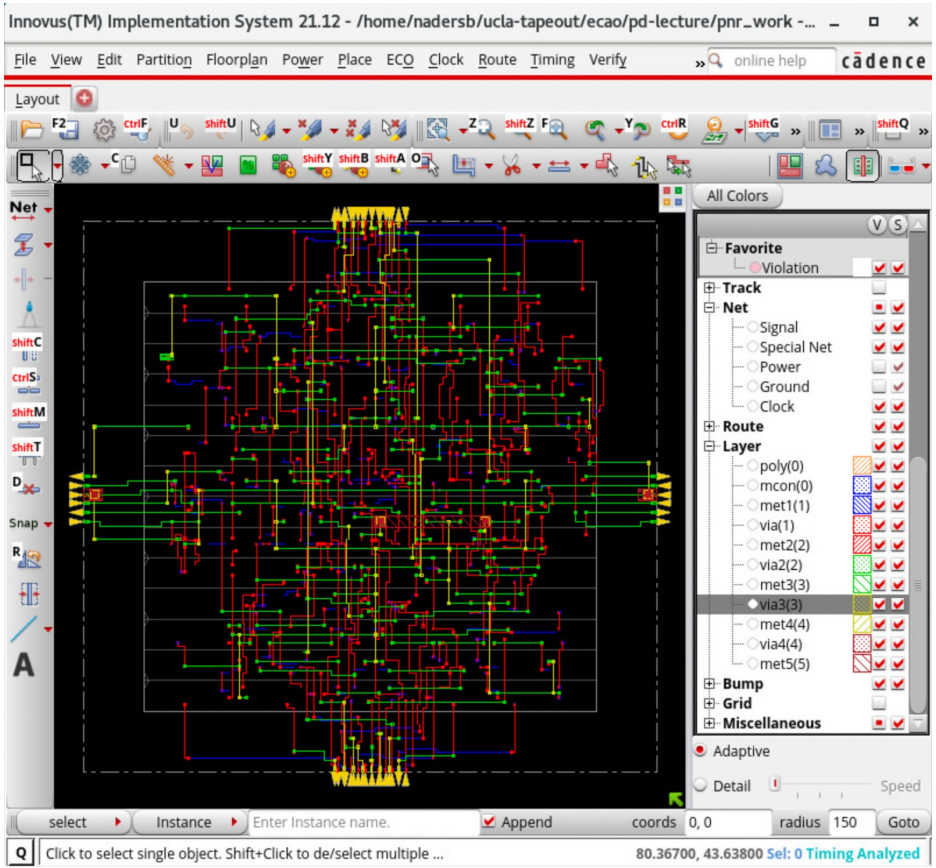

```
ccopt_design
```



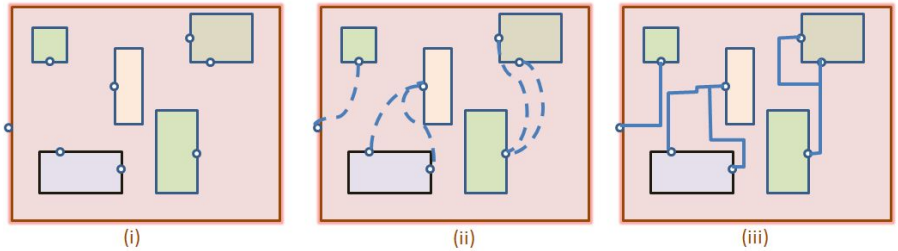


Clocks are mostly same length to reduce clock skew

Detail Route, Postroute




```
routeDesign
```



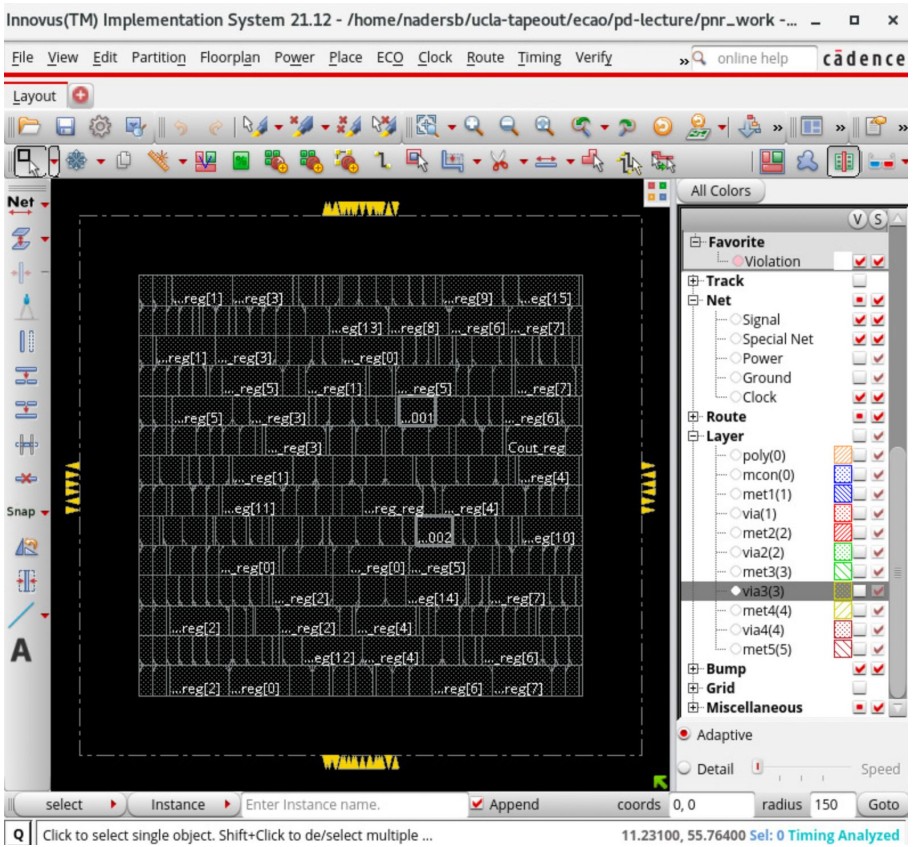
An example showing – (i) Placement of standard cells, (ii) After global routing, (iii) After detailed routing

Previous stages global route signal nets (estimated wire lengths), now we detail route.

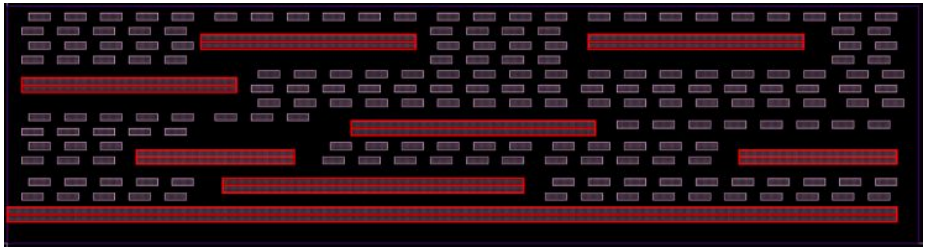
Global Route: estimation tool used to gauge congestion and wire metrics before committing to actual wire layout.

Detail Route: physically finalize the coordinates and shapes used for each signal route.

Metal Fill, Filler Cells




```
setFillerMode -core {FILL4 FILL2 FILL1}
addFiller
```

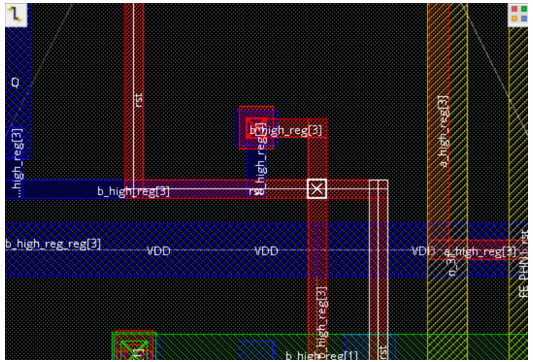


Filler Cells: populate empty space not filled by other logic or physical cells - **maintain power rail continuity, p-well & n-well continuity**

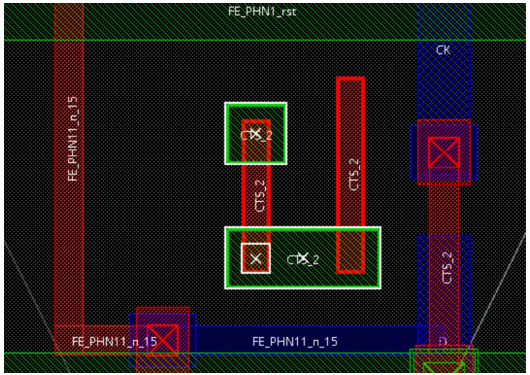
Metal Fill: connected pg or disconnected metal shapes fill empty space to **satisfy metal density requirements**

DRCs

Innovus does a lite version of DRC checking (metal), Calibre performs all the checks (base + metal)



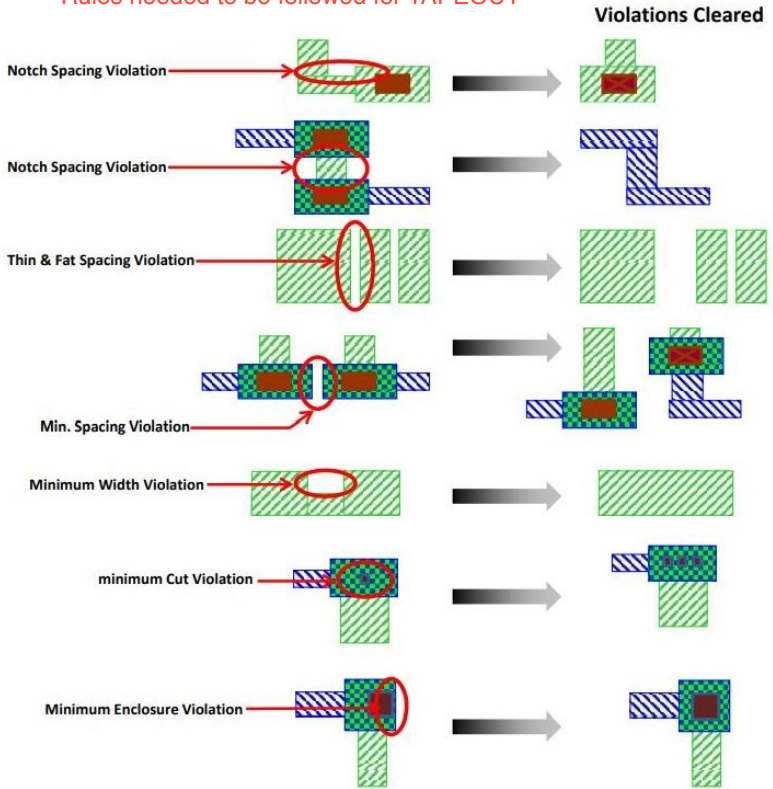
Short



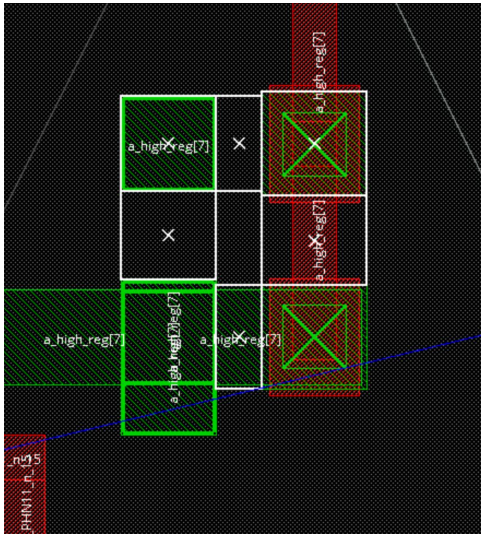
Min Area

Too Small area to be manufactured

Rules needed to be followed for TAPEOUT



verifyDrc



Min Area, Parallel Length Spacing

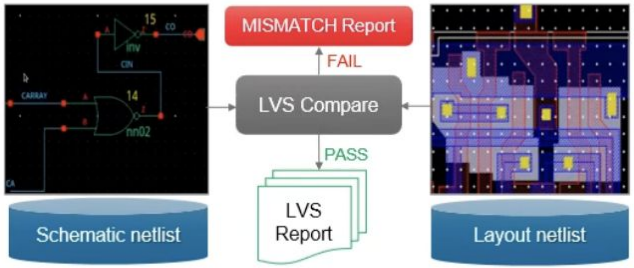
PARTICIPATE!

LVS

LVS (Layout vs Schematic) checks whether your layout matches your schematic:

Device-by-Device!

Connection-by-Connection!



- Schematic – the golden reference
- Checks devices and connectivity
- Mandatory before tapeout

```
verifyConnectivity
```

In what cases would the extracted layout netlist not match the original gate level netlist we started with?

- **Opens**
- **Shorts**
- **Unplaced cells**

IR Drop

V = IR, literally just Voltage Drops… PD is just a field of buzzwords

Various factors can cause voltage to drop in regions in the design and cause timing failure, excessive heat or power dissipation.

Weak Power Grid

Hotspots occur near macros where the PG stripes are interrupted.
Insufficient stripes means more current is concentrated in one stripe.
Insufficiently wide stripes

Takeaway: add more parallel PG paths

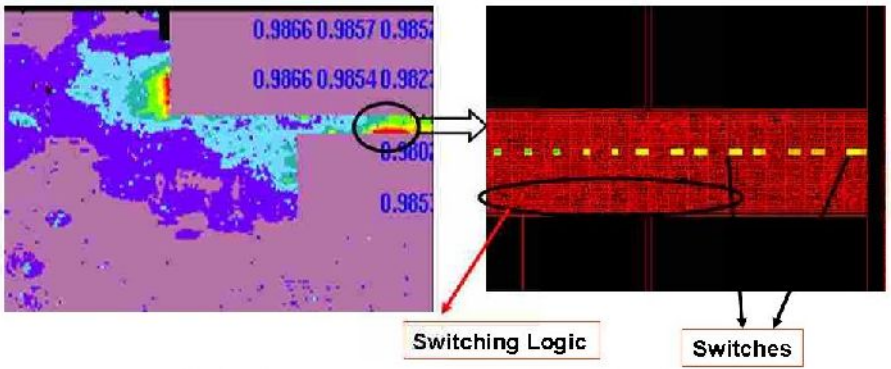
Switching Activity

High switching activity cells, like clock tree cells, switch every cycle.
Clumps of high drive strength cells (large cells, buffers) switching together.

Takeaway: use large cells with caution, space cells apart

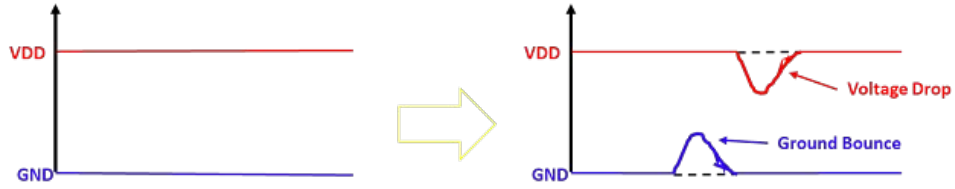
IR Drop

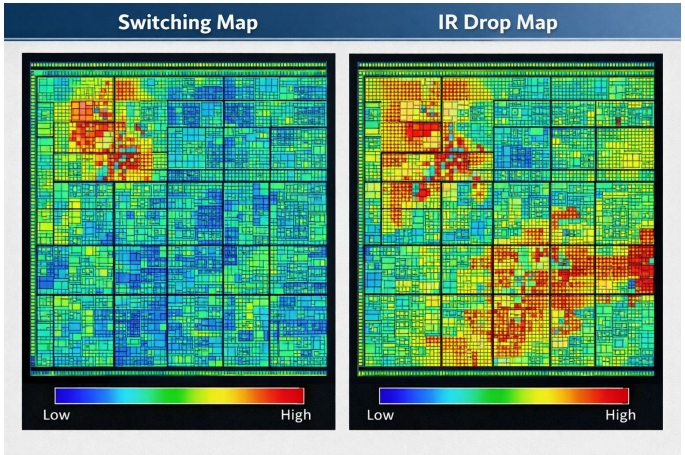
Various factors can cause voltage to drop in regions in the design and cause timing failure, excessive heat or power dissipation.



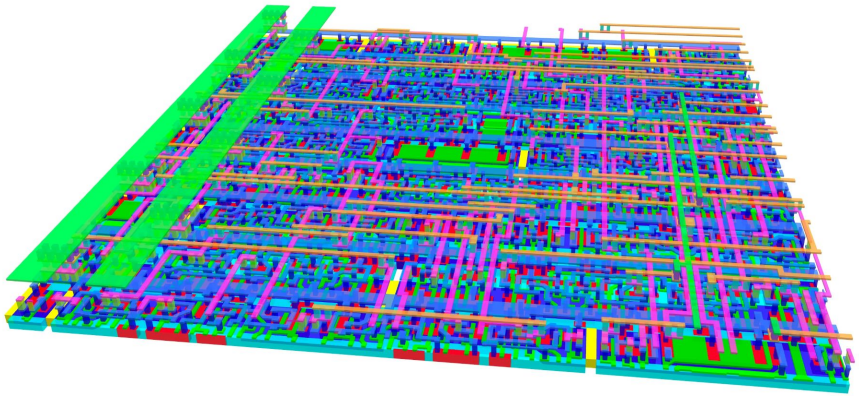
# Point	Drop (mV)
VIA34	2.6
MET3	0.0
VIA23	6.3
MET2	0.0
Switch_1	31.6
VIA23	6.3
MET3	51.4
VIA23	5.2
MET2	0.9

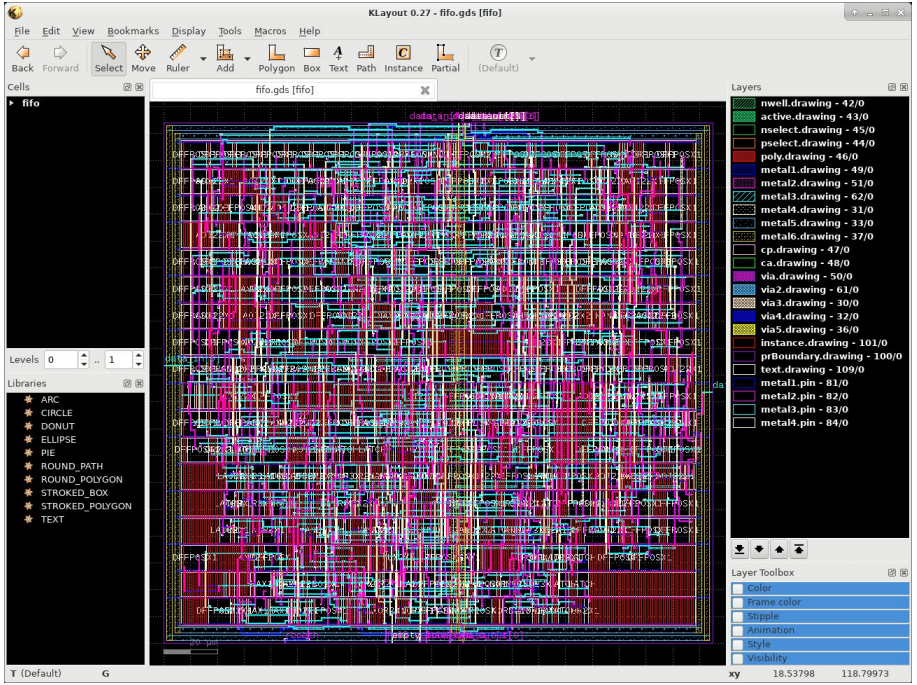
High drop due to distance from Switch





Finished GDS

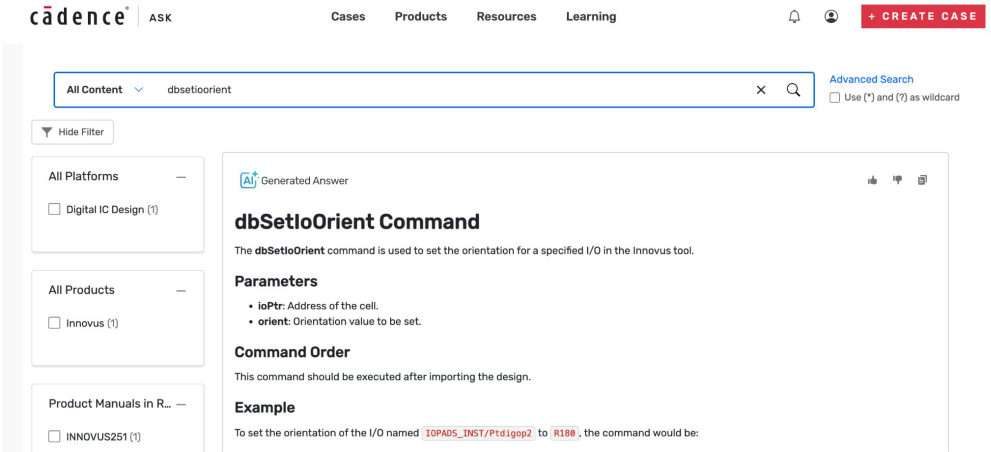


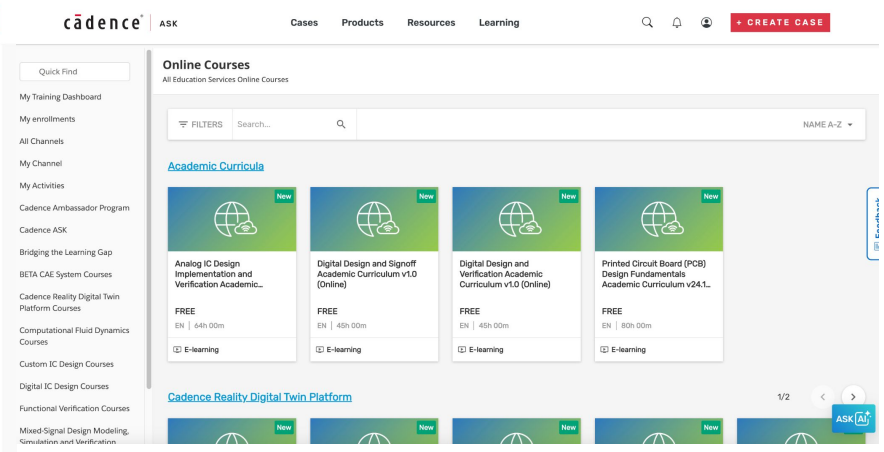


Brief Aside: Cadence ASK for Command Documentation

if you want to take Cadence certification courses or get access to Cadence resources like their user guides and such, you can do so here: https://registration.cadence.com/CadenceApplicationLoginScreen?appcode=cos&langcode=en (Create your Cadence account and verify your academic email address, Select “Academic or Content Only Access”, Enter your institution details to complete enrollment)

CAN ACCESS CADENCE COURSES FOR PD COMMANDS



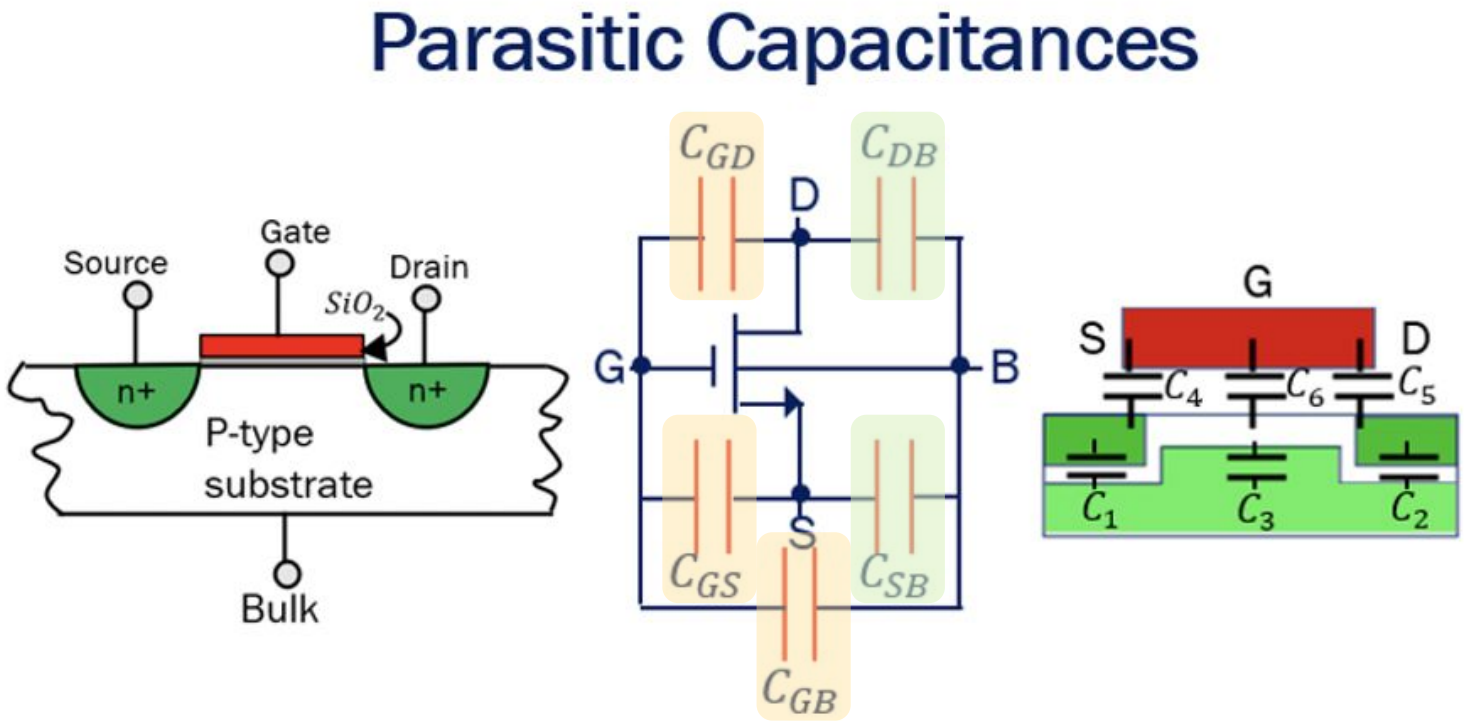


Break OR we walk to Faraday

We can now run the flow and understand the inputs and outputs, but how do we **analyze our quality of results (QoR)**?

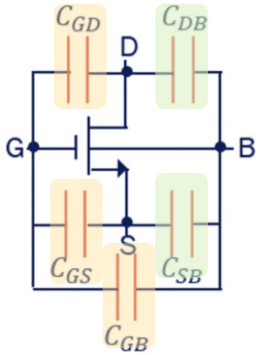
Mode of analysis number 1:

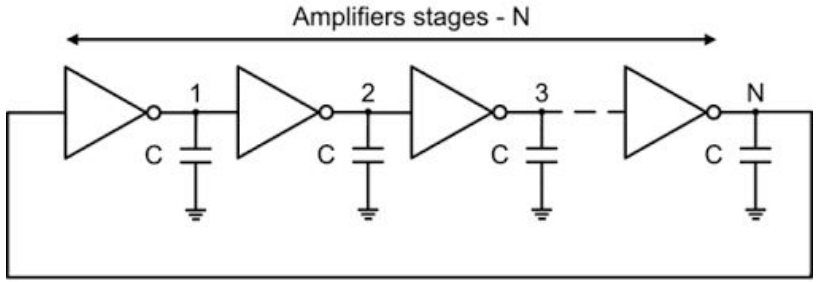
RC Constants are Delays

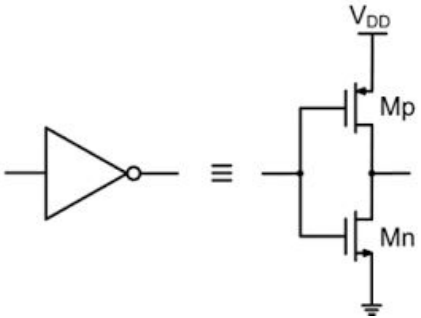


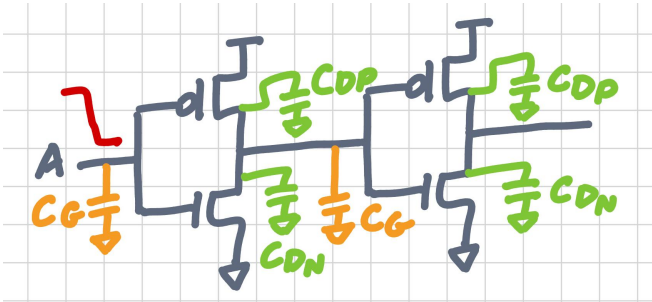
MOSFETS and transistors are imperfect by design, each have their own capacitances

RC Constants are Delays







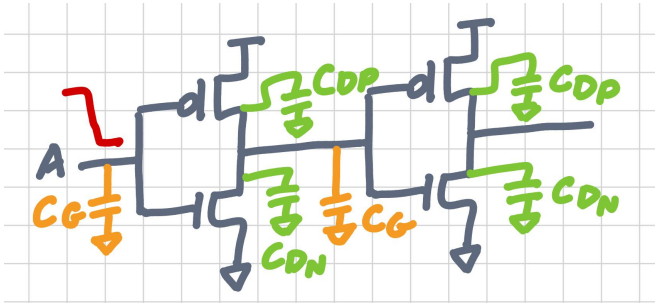


Process of charging and discharging next states
Gates are used bc they only use power when they switch, otherwise don't use as much

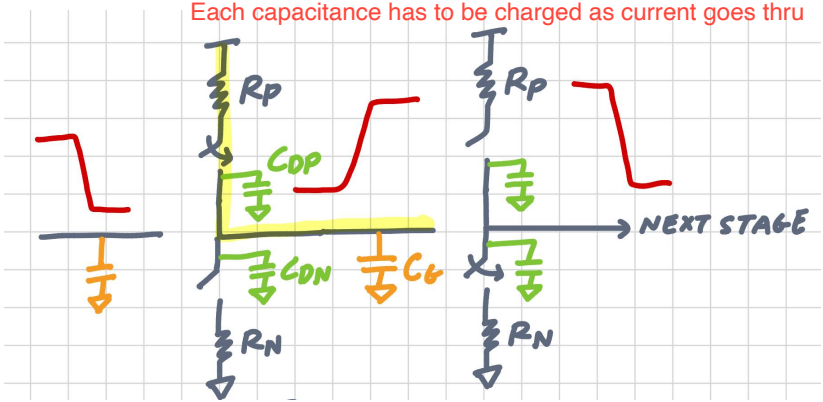
UCLA Samueli School of Engineering

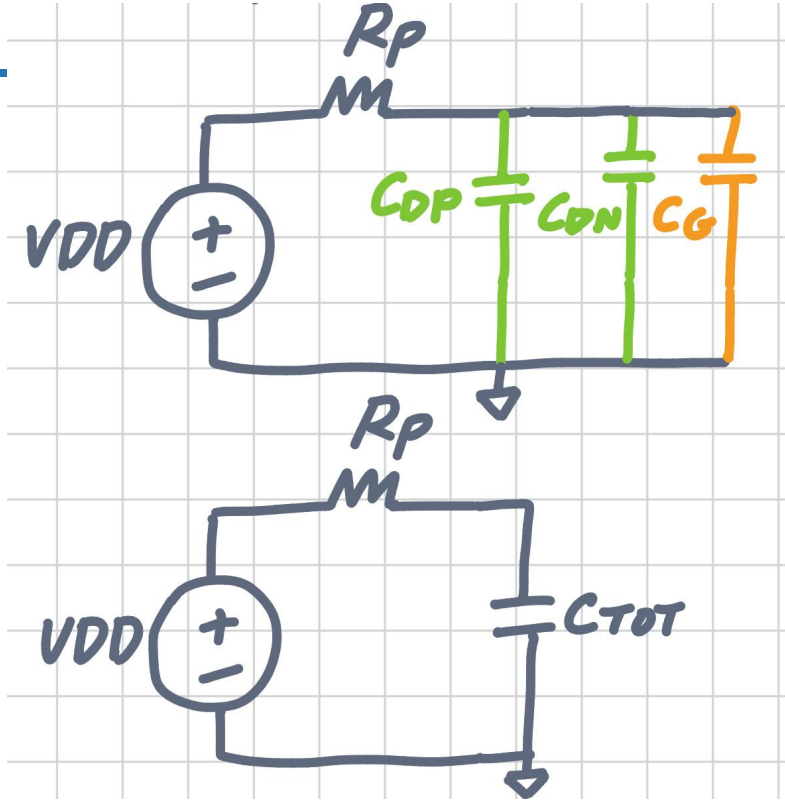
Each cell charges and discharges the next cell's gate capacitance.
Minimizing capacitance results in a reduction in delay.

RC Constants are Delays



Each capacitance has to be charged as current goes thru



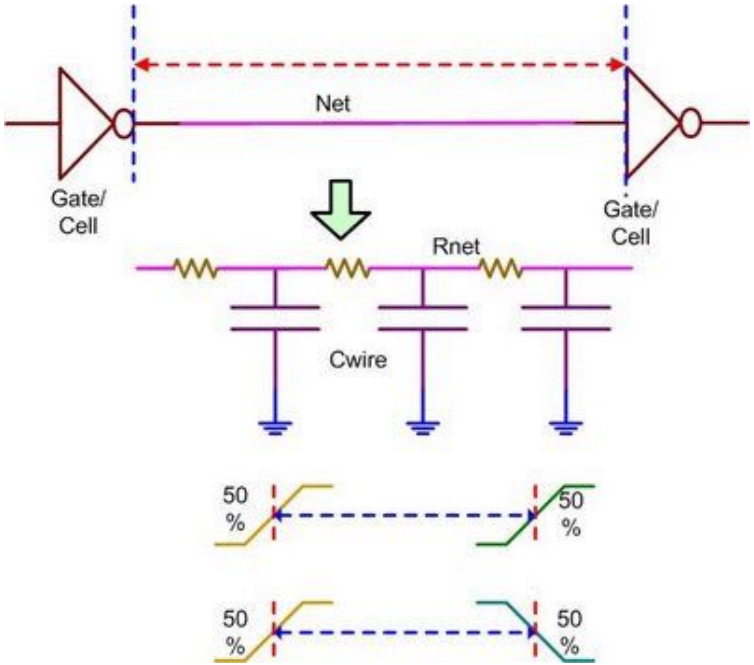


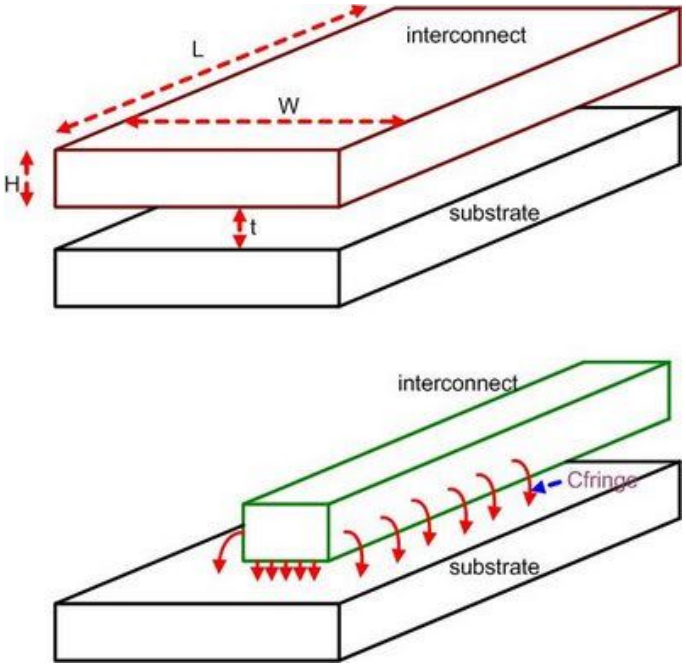
UCLA Samueli School of Engineering

Each cell charges and discharges the next cell's gate capacitance. Minimizing capacitance results in a reduction in delay.

Wire Delay >> Cell Delay

In backend design, wires are real and have non-negligible resistance and capacitance. Signals propagate through wires, which contributes to the total delay.





Knowledge Check: Cell Delay

An inverter drives another inverter. Say the second inverter is replaced by:

2 inverters in parallel (fanout 2)

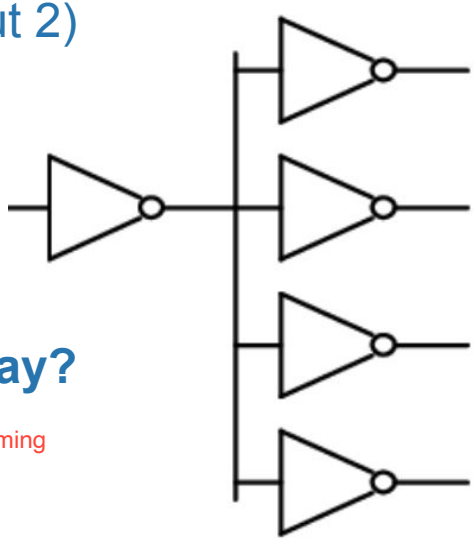
Capacitance much greater, each capacitance in parallel

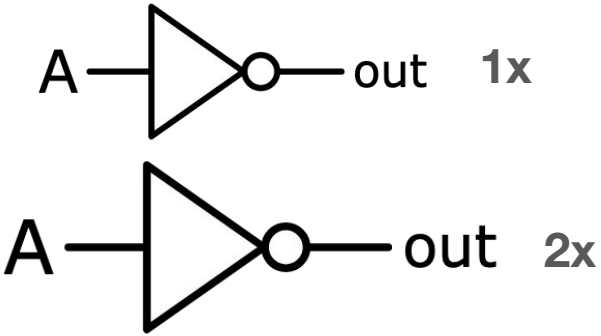
A larger inverter

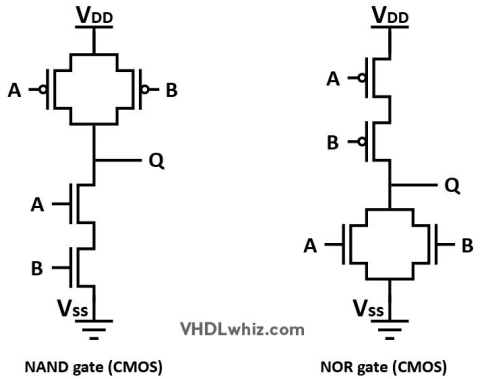
A NAND cell

Effect on propagation delay?

All of them increase capacitance, delay increased for timing

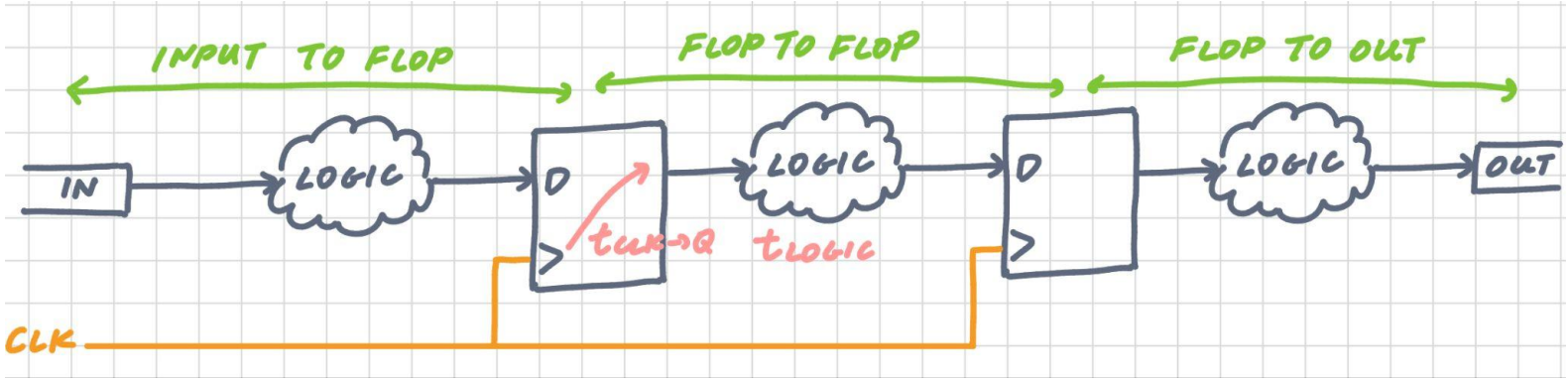


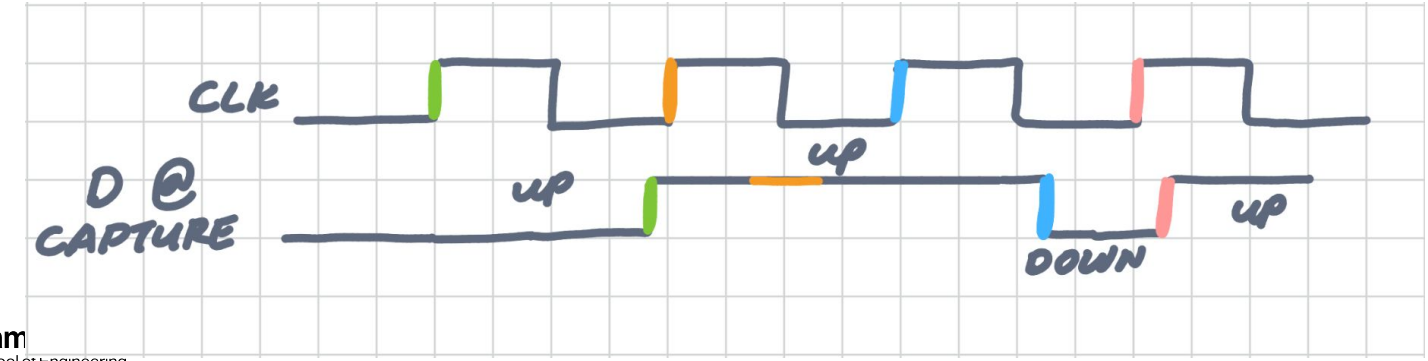




Timing

The capture flop has to arrive

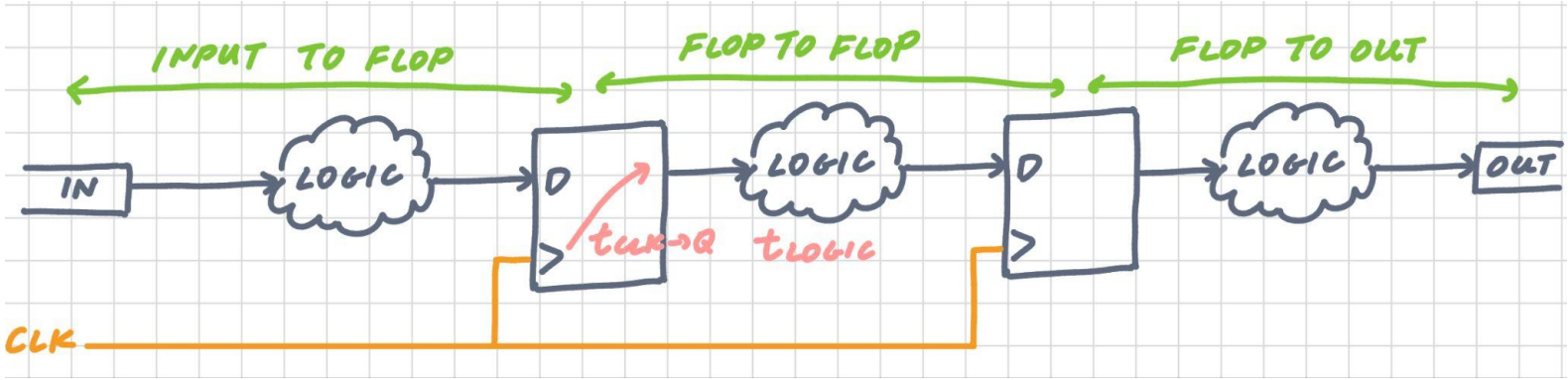


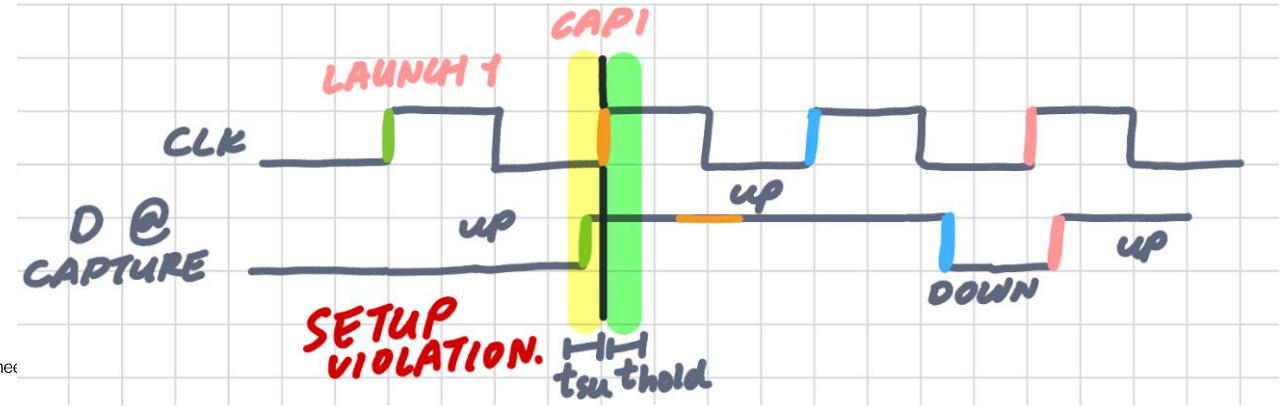


UCLA Sam School of Engineering

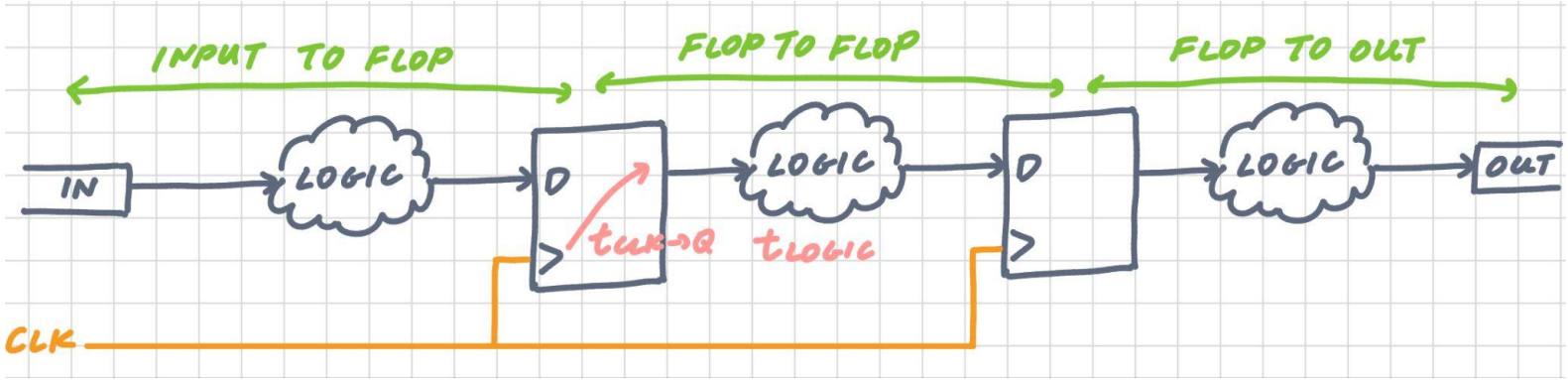
Same color means data signal was launched by that edge.

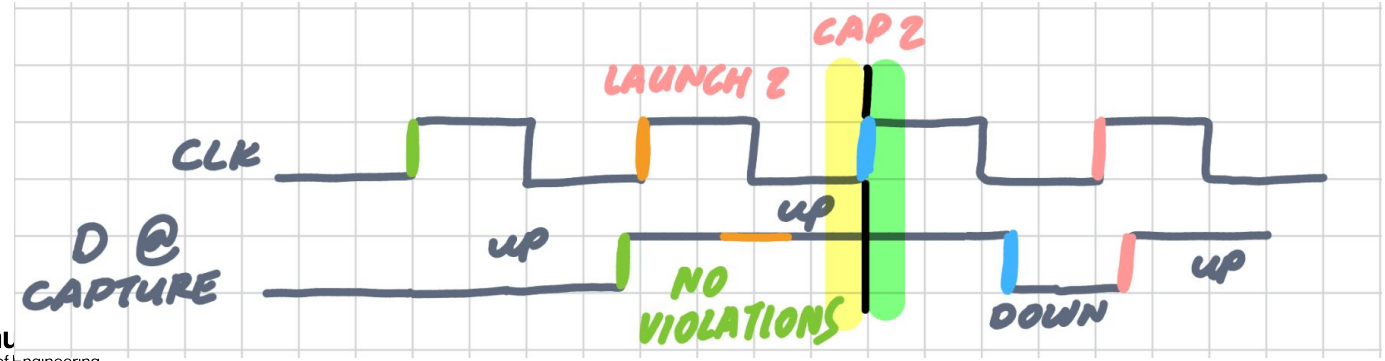
Setup Violation



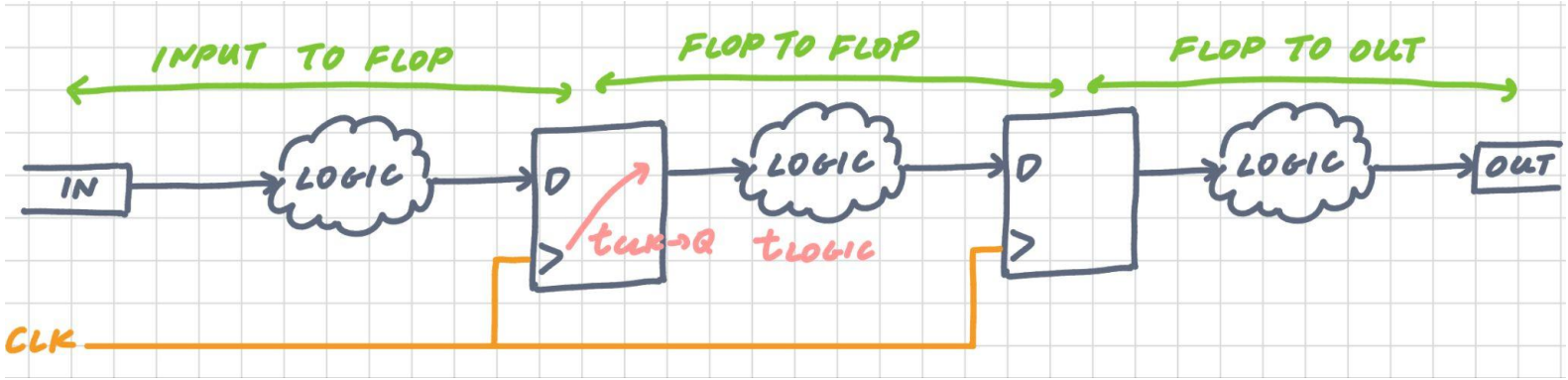


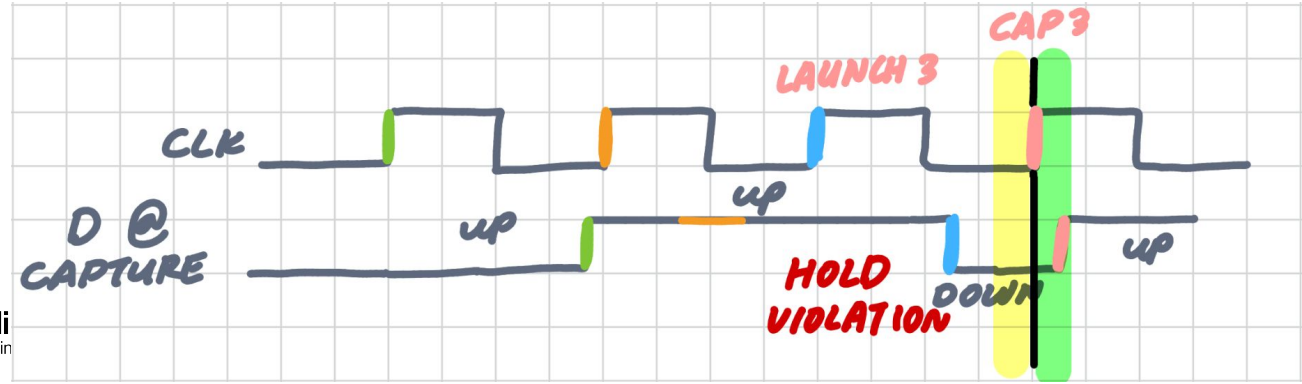
Timing Clean





Hold Violation

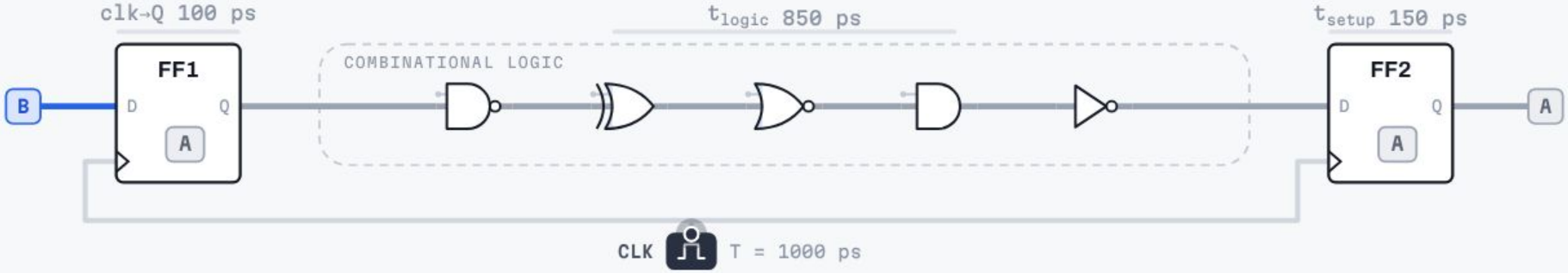


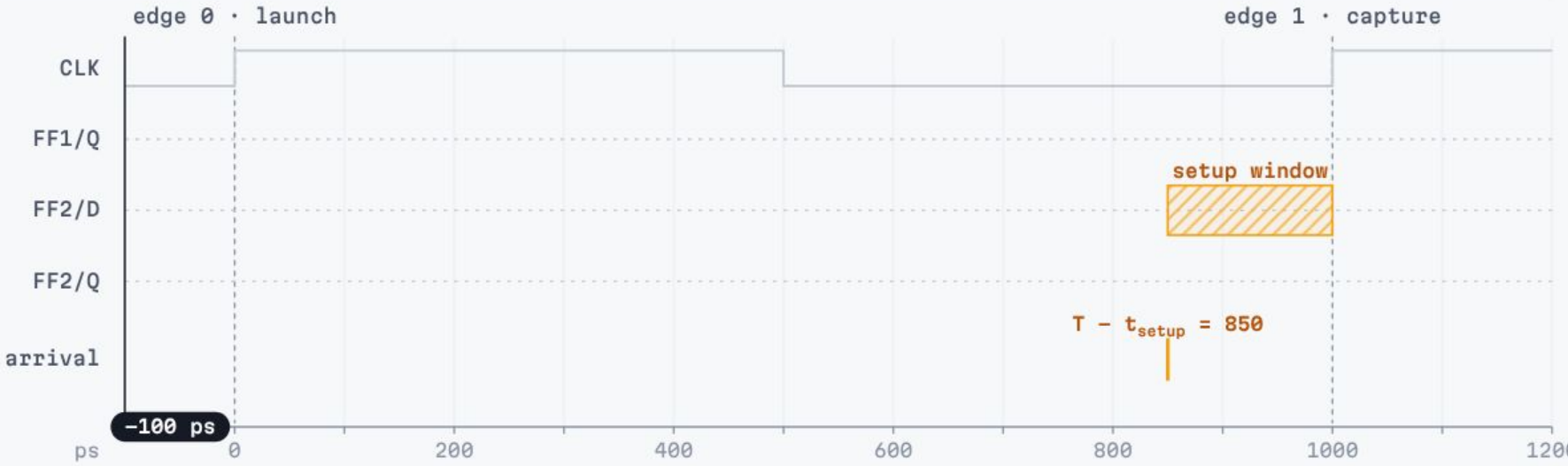


Setup violation

A old data B new data X unknown

Clock edge 0 heads to both flops.

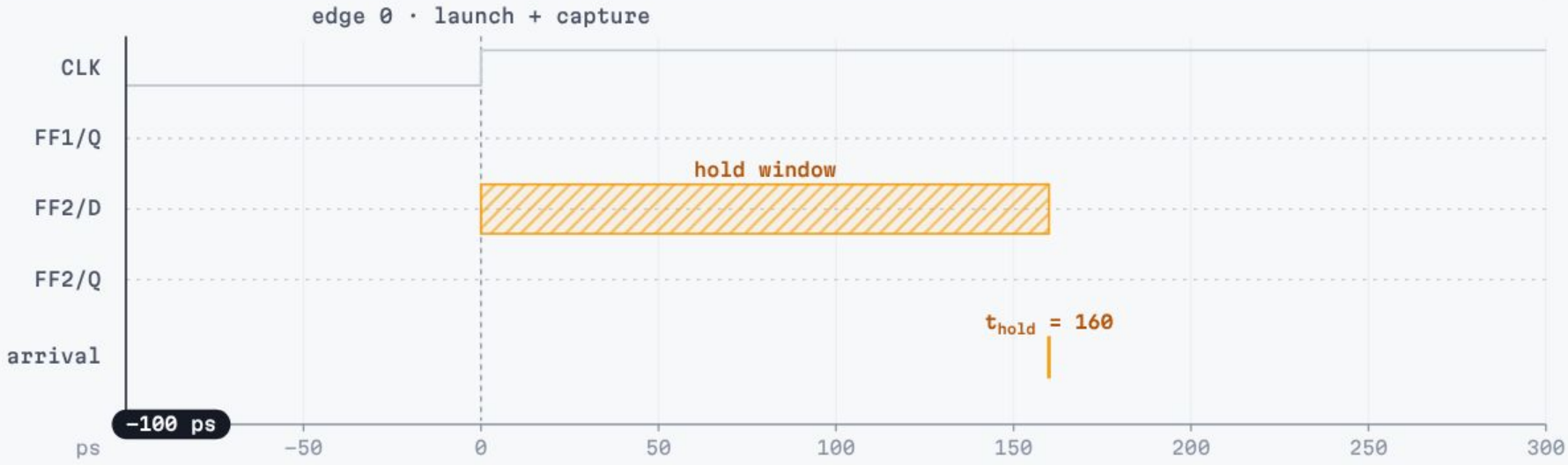




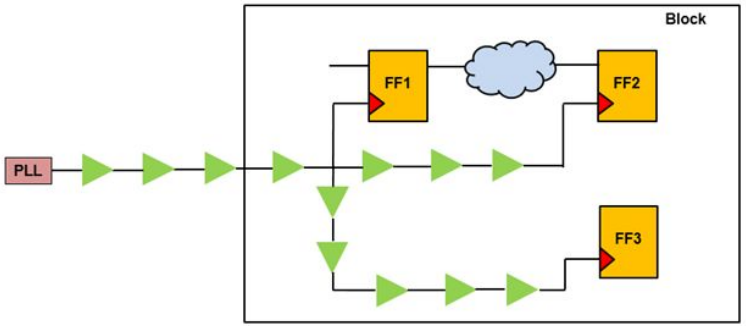
Hold violation

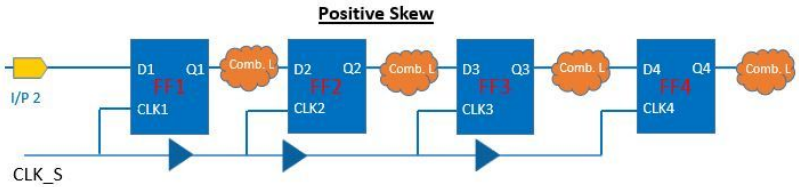
A old data B new data X unknown

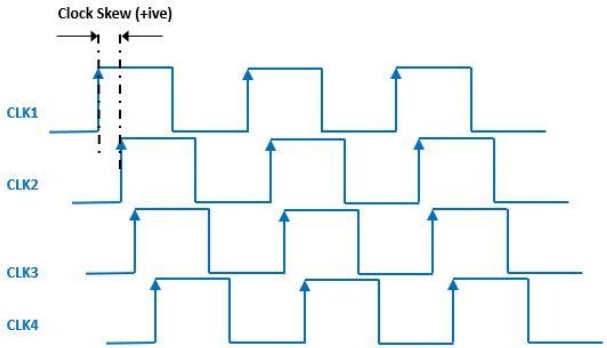
Edge 0 is about to clock FF1 and FF2 at the same instant.



Clock Skew





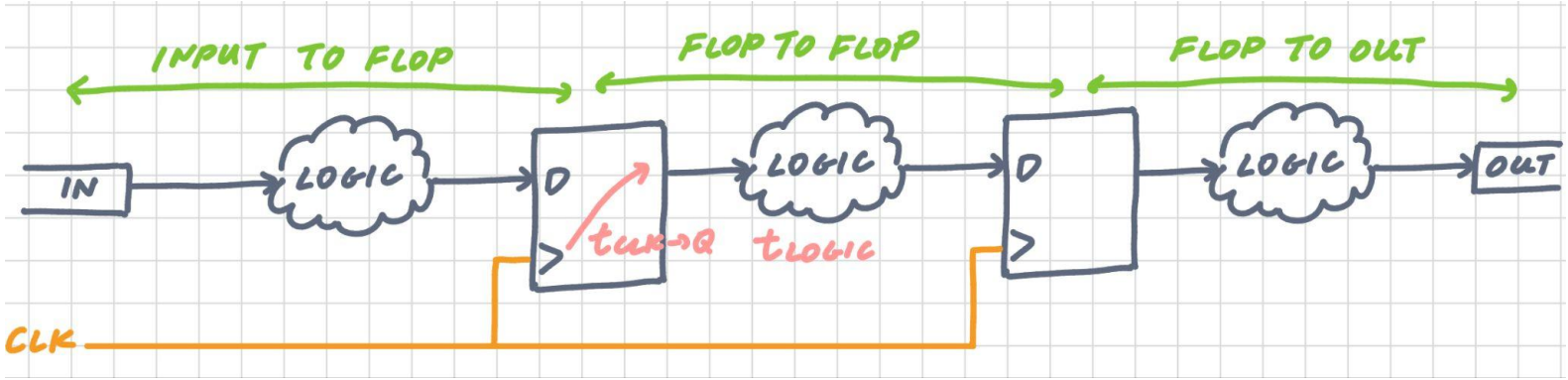


$$t_{skew} = t_{CLK->capture} - t_{CLK->launch}$$

Clock signals are delivered to all sequential elements using a clock distribution network. **Clock signal will not all arrive at the same time, leading to clock skew. Is positive skew helpful or hurtful to setup time in a timing path?**

Basically time for the next clock signal to arrive at the next ff, connected to same ffs

Timing Equations



$$T_{c2q} + T_{comb} + T_{setup} \leq T_{clk} + T_{skew}$$

$$T_{c2q} + T_{comb} \geq T_{hold} + T_{skew}$$

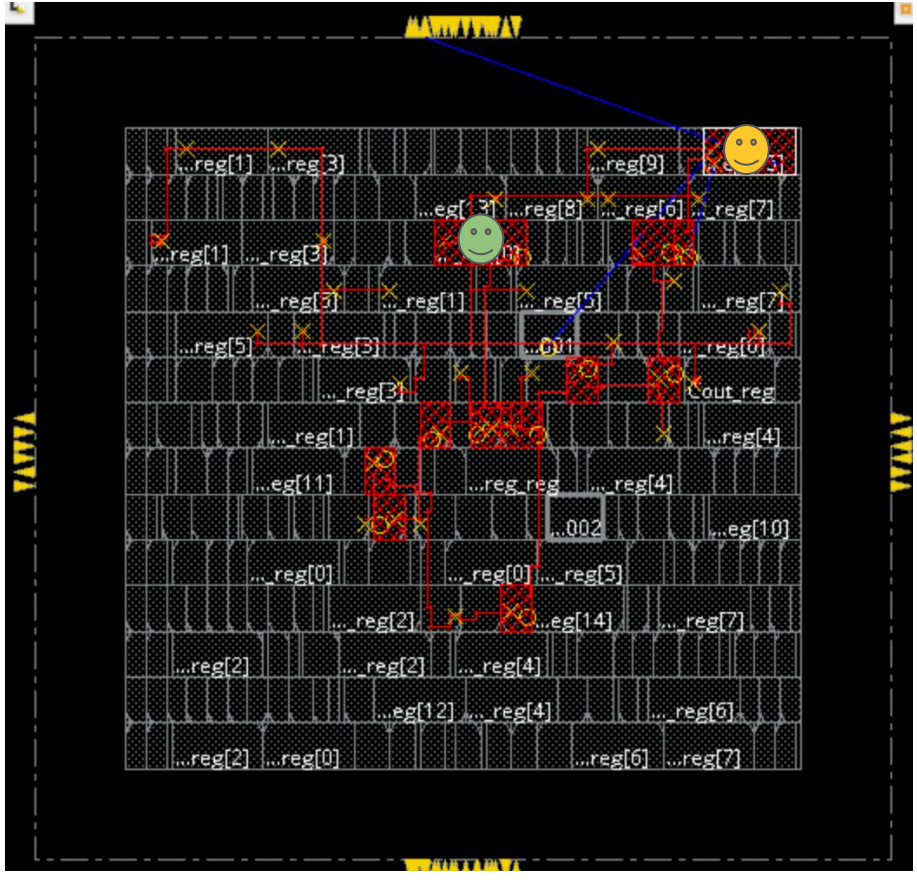
Holy Grail of setup and hold

Analyze a Timing Report

Reds are ur nets

```
innovus 62> report_timing
#####################################################################
#  Generated by:      Cadence Innovus 21.12-s106_1
#  OS:                Linux x86_64(Host ID mecals3)
#  Generated on:      Tue Oct 27 22:44:20 2026
#  Design:            RCA_16_bit
#  Command:           report_timing
#####################################################################
Path 1: MET Setup Check with Pin sum_reg[15]/CK
Endpoint:   sum_reg[15]/D          (^) checked with  leading edge of 'clock'
Beginpoint: b_high_reg_reg[0]/QN (^) triggered by  leading edge of 'clock'
Path Groups: {clock}
Analysis View: WC_SLOW_VIEW
Other End Arrival Time          0.172   Skew
- Setup                         0.177   Setup Time
+ Phase Shift                1000.000   Period
= Required Time               999.995
- Arrival Time                  3.853   Propagation, Logic Delay
= Slack Time                  996.142   Slack
     Clock Rise Edge                0.000
     + Clock Network Latency (Prop)  0.172
     = Beginpoint Arrival Time      0.172
     +-------------------------------------------------------------------------------+
     |     Instance      |     Arc      |  Cell   | Delay | Arrival | Required |
     |                   |              |         |       |  Time   |   Time   |
     |-------------------+--------------+---------+-------+---------+----------|
     | b_high_reg_reg[0] | CK ^         |         |       |   0.172 |  996.315 |
     | b_high_reg_reg[0] | CK ^ -> QN ^ | DFFX1   | 0.620 |   0.793 |  996.935 |
     | g969__8428        | A ^ -> Y v   | XNOR2X1 | 0.419 |   1.211 |  997.353 |
     | g906__1666        | A1 v -> Y ^  | OAI22X1 | 0.262 |   1.473 |  997.615 |
     | g902__9315        | A1 ^ -> Y v  | AOI22X1 | 0.276 |   1.748 |  997.891 |
     | g898__7482        | A1 v -> Y ^  | OAI22X1 | 0.284 |   2.033 |  998.175 |
     | g894__6131        | A1 ^ -> Y v  | AOI22X1 | 0.300 |   2.333 |  998.475 |
     | g890__8246        | A1 v -> Y ^  | OAI22X1 | 0.386 |   2.719 |  998.861 |
     | g886__1617        | A1 ^ -> Y v  | AOI22X1 | 0.326 |   3.045 |  999.187 |
     | g882__6783        | A1 v -> Y ^  | OAI22X1 | 0.323 |   3.368 |  999.510 |
     | g879__6260        | B ^ -> Y v   | XNOR2X1 | 0.365 |   3.733 |  999.875 |
     | g877__5107        | B v -> Y ^   | NOR2X1  | 0.120 |   3.853 |  999.995 |
     | sum_reg[15]       | D ^          | DFFX1   | 0.000 |   3.853 |  999.995 |
     +-------------------------------------------------------------------------------+
```

Margin before ur next slack



Setup Fixing

Root cause: data is arriving too late to be captured by the capture flop.

Why do setup violations happen?

- Long wires slow down signals
- Small drivers can't switch fast enough
- Driver driving too large of a fanout
- Clock skew is really bad for a path

What can we fix on our end?

- Tweak logic and macro placement
- Upsize or swap cells to larger variants
- Buffer nets with large fanouts or restructure the logic
- Change timing constraints (clock frequency, skew groups) or floorplan to alleviate skew

What's the cost? Of upsizing a cell? Of adding buffers? Of changing the clock frequency?

bigger cell takes more power and area

lower freq = worse chip

Hold Fixing

Root cause: data is arriving too early for the previous value to be held stable by the capture flop.

Why do hold violations happen?
- Wires are too short
- Logic depth is very simple
- Positive skew

What can we fix on our end?
- Insert buffers or delay cells to increase delay
- Space target and capture flop farther away from each other intentionally
- Timing constraints fix

Literally just make the wires longer

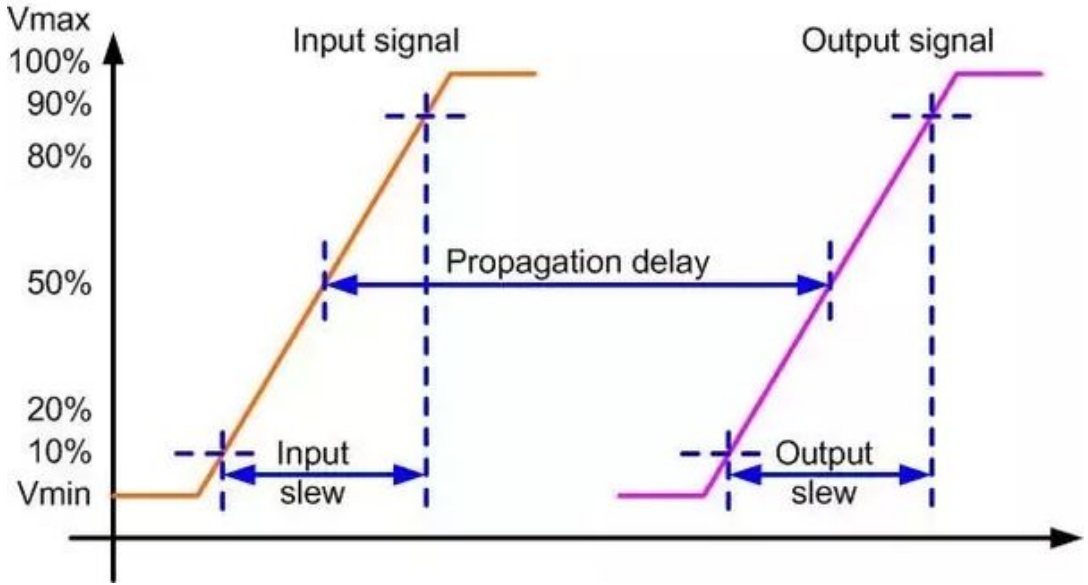
What's the cost?

Transition Violations

A cell is unable to drive the proceeding stage properly, causing the output transition to be too slow.

What can this be caused by?

- **Too high fanout**
- Too small drivers
- Long wires



Knowledge Check: Revisiting the App Question

Which is more serious, setup or hold? For PD? For validation?

In Validation, hold is more serious; for setup u can change ur clock timings, but hold u cant change ur logic

For PD, move things more farther apart, hold violation easier to do; but for setup in PD, we need to change logic

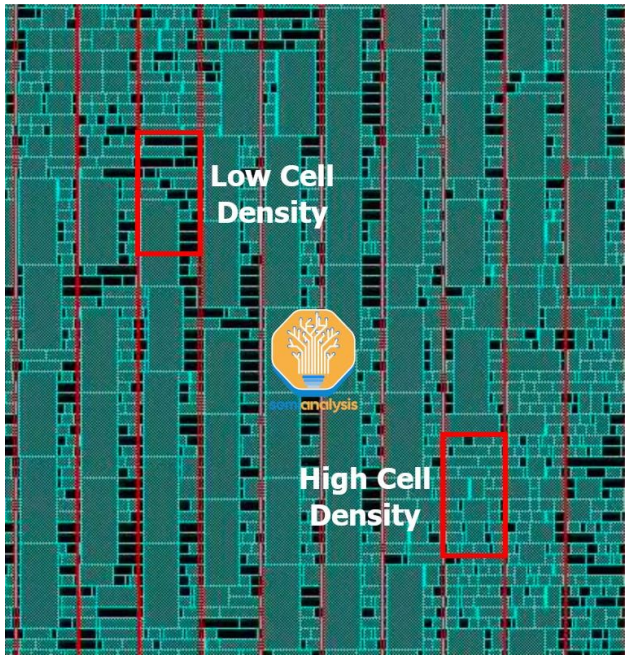
Mode of analysis number 2:

Utilization / Cell Density

Utilization: cell area over total available placeable area - how well are you utilizing your space?
Cell Density: within sections of the design, is the density of cells exceeding some threshold - are cells adequately spread apart?

Shouldn't we use 100% of our logic?

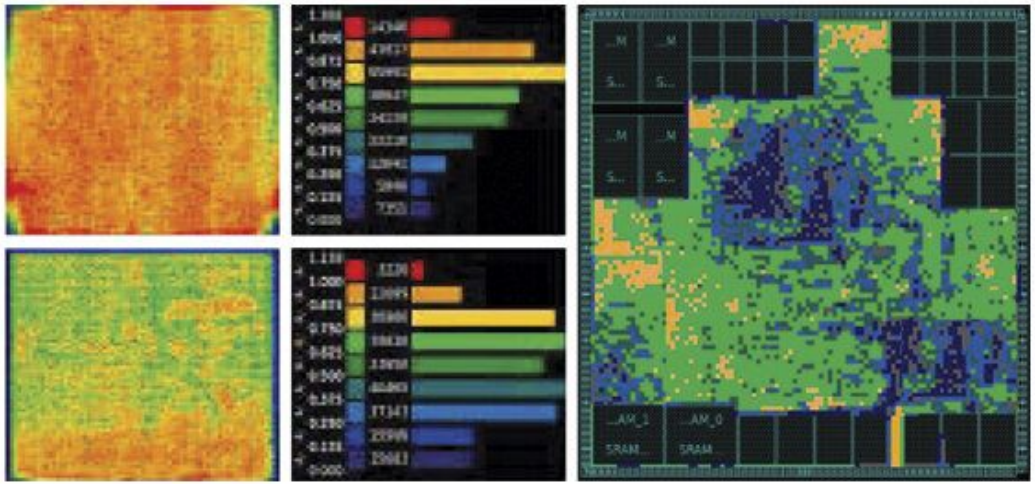
No, you are prone to IR Drop

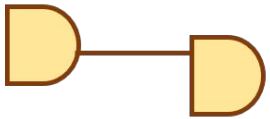


Congestion

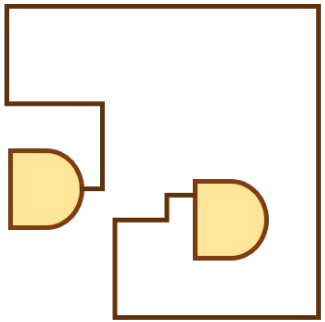
A design is divided into grids, where each grid has some finite routing resources or tracks on each layer. **Congestion is determined by taking the # of free tracks / # of available routing tracks in a single grid.**

Takes number of tracks that are routed or with free tracks, colors showing routing density; consequences because longer paths = setup violations, timing violations (propogation delay)





Ideal scenario



Practical scenario in cases with high net density

Sheer Quantities

Number of Standard Cells

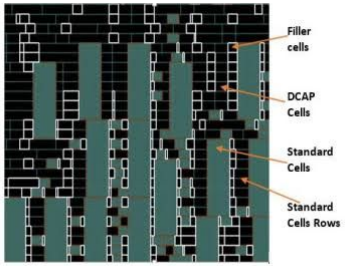
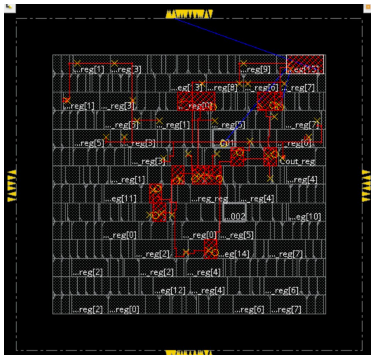
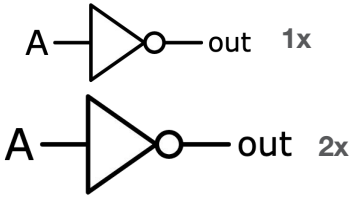


fig2: Filler cells, Decap cells

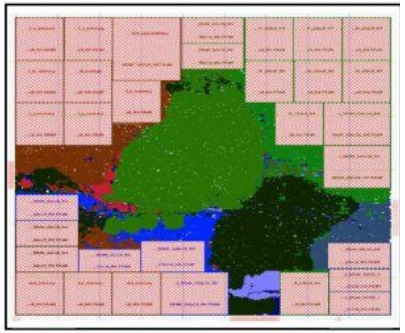
Wire Length



Distribution of VT Cells, X Sized Cells

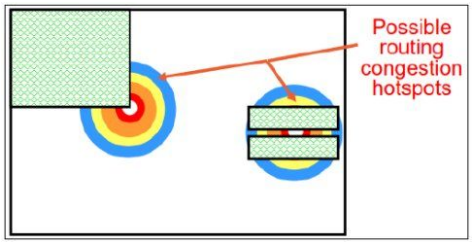


Logic Group Distribution



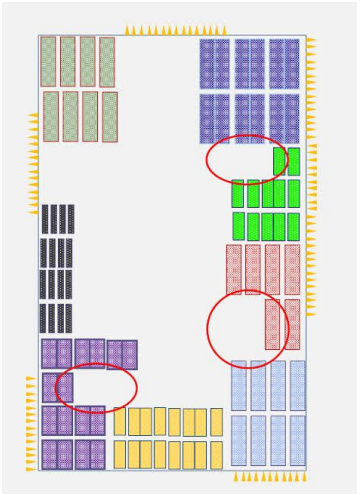
Methods for Fixing

Placement and Routing Blockages, Routing Guides



Prevent cells in areas

Floorplan Resizing, Floorplan Tweaking / Moving Macros or IO Ports



Also have partial blockages

Memories on the opposite sides of each others, they have to talk to each other, longer paths; tons of different macros

Revise Timing, Power Constraints

Not super recommended

You, probably: this sounds boring asf

let's put it all together!

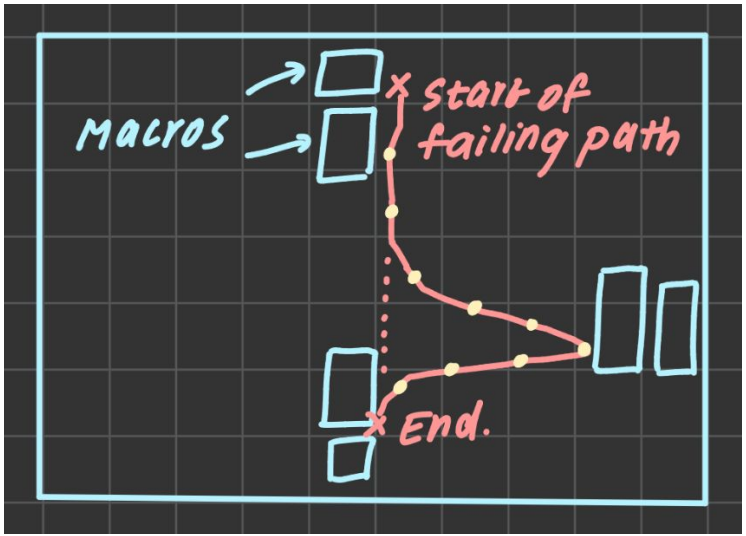
How this will (ideally) work: I give you some bad metric we need to diagnose. You ask me probing questions in the provided categories so you can come to a conclusion.

Case Study #1

Result: poor setup timing in a design. What to do?

TNS = total negative slack (sum of all data point), FEP = failing endpoints (#)

Whole Design:
TNS: 20 ns, WNS: 0.20 ns, FEP: 200
Logic Group #1:
TNS: 18 ns, WNS: 0.20 ns, FEP: 150
Logic Group #2:
TNS: 1 ns, WNS: 0.04 ns, FEP: 45
Logic Group #3:
TNS: 1 ns, WNS: 0.10 ns, FEP: 5



Detoured Path, the tool did this; Why?

Trying to connect all 3 together because of the combinational logic (yellow dots)

Case Study #1

Result: poor setup timing in a design. What to do?

Logic Groups: one logic group is responsible for the poor timing.

Failing Endpoints: of those failing paths, many start at the top of the block and end at the bottom.

Wire Length: not only that, but they detour significantly to the right in the middle of the block.

The detour is due to optimization cells being shared with the violating timing paths and a prominent logic group on the right side of the block. **Tell the tool to make these paths not share.**

Case Study #2

Shorts occur at a corner of a large macro > congestion hotspots cause nets to be displaced and to short > many long timing paths exist that must route around the macro corner

SOLUTIONS?

server&NDA

IMPORTANT PRIVACY WARNINGS

Server Access

https://forms.gle/v2RKJmaydEqskayHA

NDA PDK Access will be granted upon subteam assignment completion (any).

UCLA Samueli School of Engineering

Supplemental Resources & Recommended Classes

Classes:

Classes at UCLA which teach related content (early -> late into degree):
CS M51A, CS M152A, ECE 115C, ECE M116C, ECE M216A

THANK YOU!!

Addendum: Tool Knowledge

In progress …